卷首语

1、docker 原理及在运维工作的地位和作用

运维工作进化论，docker、微服务、k8s 的联系，devops 和 docker 的关系，docker 的前世今生。

2、容器，镜像和仓库

容器和虚拟化，优势和劣势，底层核心；
容器除了 docker 还有什么其他选择？
docker 的安装及三大核心：容器、镜像、仓库。

3、docker 的实际运用

docker 的常用命令及注意事项，镜像的原理，dockerfile 的作用和应用，docker 的永久存储和网络通信。

4、docker 实际工作案例实现

搭建一台私有仓库，镜像仓库的上传和拉取，管理仓库，微服务的概念，用 docker 实现一个实际案例。

试用水印

目录

docker 原理及在运维工作的地位和作用 **5**
一、运维工作进化论测试环境和生产环境 6
二、运维生产环境的发展 12
三、docker 概念 14
四、docker 与容器区别 14
五、进程级封装概念 14
六、微服务生产环境 15
七、k8s－容器编排管理工具 16
八、云原生技术栈的概念及技术 17
九、容器介绍 18
十、容器和虚拟化介绍和比较 18
十一、答疑环节 22

容器、镜像和仓库 **25**
一、Docker 底层技术概述 25
二、访问隔离案例 26
三、Docker 版本介绍 27
四、Docker 安装部署 28
五、docker 安装与启动 29
六、Docker 常用命令解释 30
七、docker 基础命令 40

docker 的实际运用 **44**
一、docker 基础命令 45
二、案例一 46
三、案例二 49
四、Dockerfile 52
五、dockerfile 操作测试 56
六、dockerfile 案例 1 58
七、dockerfile 案例 2 60

docker 实际工作案例实现 **63**
一、数据持久化 64
二、harbor 仓库 68
三、微服务 74

试用水印

docker 原理及在运维工作的地位和作用

演讲人：马哥教育李老师 阿里云 IoT 技术专家

视频链接 ：https://developer.aliyun.com/learning/course/892/detail/14271?spm=a2c6h.21258778.0.0.74b6db0d28oJyG

各位学员大家晚上好，首先，还是欢迎来大家来到这个训练营，今天是第一天，这个主题叫让我们对 docker 有一个全新的认识。为什么我敢用这个全新的认识？有可能你们已经做了有三五年的运维了，有可能你们年薪已经 30 万以上了，对吧，但是相信我你们听完今天的这堂课还是有一些原来不太了解的事情，然后进入到你的认知里边。大致今天讲的内容就是包括运维工作的进化论，docker 的 devpos、 k8s、微服务的关联等，docker 是否可以被替代，docker 的前世今生就是内容，简单地做一个自我介绍，叫我李老师也行，叫我泽辰老师也行。这个是我们整个四天课程的大致的说明，首先，课程内容是四天每天两小时内容是比较多的，第二个课程是全程直播的，课程资料是课后打包会给到大家。我们是有奖品的，不只是免费。并且还会给到你们一共五本《kubernetes 进阶实战第二版》作者马永亮，马永亮在运维界，在技术上还有教学上都是非常厉害的一个人。

下边课程中会给大家一点时间去进行操作，这个也是基于我之前讲课，一定会给到大家一点时间，到时候尽量去做。请各位学员跟着我的思路去走，因为第一天第二天都没什么难度，第三天难度就直线拔高，第四天，同样难度也是比较大的。所以说，尤其在第三天第四天的时候，一定要跟着我的思路去走，否则的话有一点你没有跟上，或者是有一个知识点你忽略了，那你可能在你这个操作的过程中，就会出现很多的问题，推荐你们在思考的同时把这个步骤记下来可以加深印象。最后放平心态，为自己赚一个好未来，这句话我在以往给我所有的学员去说的时候都会告诉他们，我自己在学习的时候包括现在如果有一些想要去了解更深刻的一些技术点技术站的话，我也会把这句话跟自己说，所以说技术都是需要不断进步不断去学的。接下来正式进入我们的正课。然先和大家说一下，我们第一天打卡有三个东西，第一个是一个 cent S7 点六还是七点九的一个镜像，还有个 vmware 的 CDK 你们把这三个东西都下来，并且安装好，大家都是已经是有一定基础的，否则不会过来听 docker，你们需要自己把这个搞好统一环境，就用我那个镜像 vmware。这个是和大家非常强调的一点。

这节课主要围绕十一个部分来讲，首先我们来讲一下运维工作进化论测试环境和生产环境。

一、运维工作进化论测试环境和生产环境

1.测试环境

a.一套 LNMP 应用

LNMP 是一套技术的组合，L=Linux、N=Nginx、M=MySQL、P=PHP

Nginx 服务是不能处理动态请求。

静态请求：请求静态文件或者 html 页面，服务器上存在的 html 文件（静态文件，指上传时什么样子，访问时还是什么样子）

动态请求：请求的是动态内容，带参数的请求。动态页面不存在于服务器上，它可能是取 2 数据库或者 redis 等地方取值拼凑成的页面。

当用户发起 http 请求，请求会被 Nginx 处理，如果是静态资源请求 Nginx 则直接返回，如果是动态请求，Nginx 则通过 fastcgi 协议转交给后端的 PHP 程序处理。

试用水印

b.一台服务器。

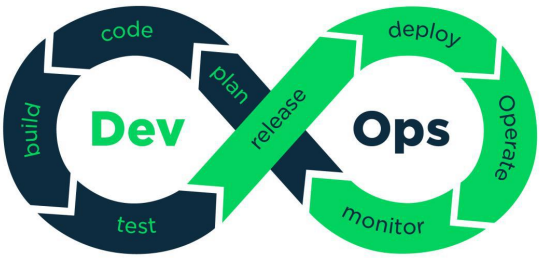
c.更新用 ftp 更新。

FTP 有着极高的延时，这意味着，从开始请求到第一次接收需求数据之间的时间，会非常长；并且不时地必须执行一些冗长的登录进程。

2.解决的问题

- 上生产后有了一定程度的流量，需要运营和测试等部门给出数据，根据数据考虑冗余。
- 防止单点故障和保证业务稳定，做 HA 集群。
- 成本角度考虑，先使用公有云服务器，提前部署。
- 更新方式太过原始，测试环境开始先改用 git。并搭建使用 gitlab 或 gogs。

3.原生产环境

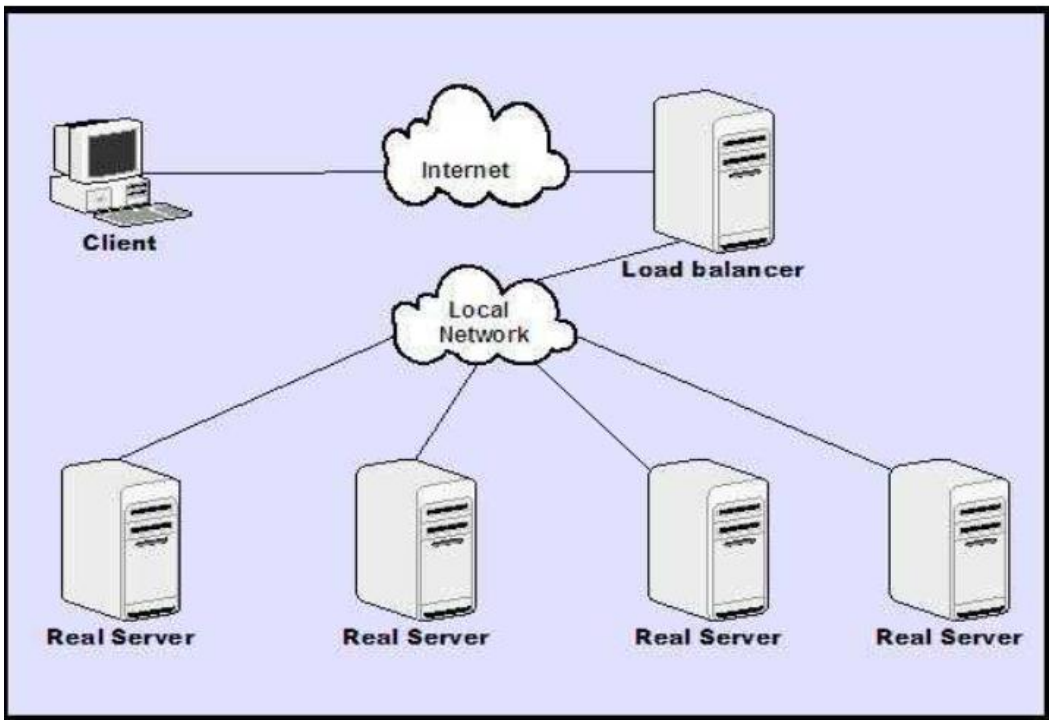


a.多点集群单体应用

b.公有云

一种按使用量付费的模型，用户可以随时随地、便捷地、按需地从可配置的计算资源共享池中获取所需的计算资源（网络、服务器、存储、应用程序等服务），这些资源可以快速供给和释放，用户只需投入较少的管理工作。公有云：云计算运营商拥有超大规模基础设施，对外提供云服务。

c.git 更新



试用水印

多点集群单体应用，把它分成两块，一块是多点集群架构图，这是附带均衡器（Load balance）下边是 Local Network，Local Network 里面有包括真实服务器（Real Server），也就是我们的原站。这是多个服务器，多个点，共同为一套代码进行服务。

多点集群就是很多服务器在服务，服务的是一个单体应用，架构里面，服务的单体应用就是服务的某一个应用，这四个服务器里面，原站里面都是一套代码。

4.虚拟化和云计算



1）物理隔离

a.概念：

指采用物理方法将内网与外网隔离从而避免入侵或信息泄露的风险的技术手段。物理隔离主要用来解决网络安全问题的，尤其是在那些需要绝对保证安全地保密网，专网和特种网络与互联网进行连接时，为了防止来自互联网的攻击和保证这些高安全性网络的保密性、安全性、完整性、防抵赖和高可用性，几乎全部要求采用物理隔离技术。物理隔离包含隔离网闸技术 、物理隔离卡等。

物理隔离产品是用来解决网络安全问题的。尤其是在那些需要绝对保证安全地保密网，专网和特种网络与互联网进行连接时，为了防止来自互联网的攻击和保证这些高安全性网络的保密性、安全性、完整性、防抵赖和高可用性，几乎全部要求采用物理隔离技术。

b.特点：

- 资源利用率极低
- 灵活性差
- 成本高

2）虚拟化的技术

通过硬件和软件支持实现物理架构资源的重新整合利用，可以用一台物理机，通过虚拟化就可以划分为多套系统，然后在系统之内进行一个多方位的隔离，隔离以后就相当多台服务器。

虚拟化是资源的一种逻辑表示，并不会受限于物理资源。运行的环境不在真实的硬件上，而是硬件之上的虚拟内存中的一段，或者说是虚拟的环境中。虚拟化为数据、计算能力、存储资源等提供了一个逻辑视图，而不是物理视图。虚拟化的发展，大幅降低了 IT 硬件成本，减少了资源的浪费，并且提升了系统的稳定性和安全性。

试用水印

1）云计算（概念）

a.概念

分布式计算技术的一种，其最基本的概念，是透过网络将庞大的计算处理程序自动分拆成无数个较小的子程序，再交由多部服务器所组成的庞大系统经搜寻、计算分析之后将处理结果回传给用户。透过这项技术，网络服务提供者可以在数秒之内，达成处理数以千万计甚至亿计的信息，达到和“超级计算机”同样强大效能的网络服务。

B.特点

b. 超大规模

“云”具有相当的规模，Google 云计算已经拥有 100 多万台服务器， Amazon、IBM、微软、Yahoo 等的“云”均拥有几十万台服务器。企业私有云一般拥有数百上千台服务器。“云”能赋予用户前所未有的计算能力。

c. 虚拟化

云计算支持用户在任意位置、使用各种终端获取应用服务。所请求的资源来自“云”，而不是固定的有形的实体。应用在“云”中某处运行，但实际上用户无需了解、也不用担心应用运行的具体位置。只需要一台笔记本或者一个手机，就可以通过网络服务来实现我们需要的一切，甚至包括超级计算这样的任务。

d. 高可靠性

”云”使用了数据多副本容错、计算节点同构可互换等措施来保障服务的高可靠性，使用云计算比使用本地计算机可靠。

e. 通用性

云计算不针对特定的应用，在“云”的支撑下可以构造出千变万化的应用，同一个“云”可以同时支撑不同的应用运行。

f.高可扩展性

“云”的规模可以动态伸缩，满足应用和用户规模增长的需要。

g.按需服务

“云”是一个庞大的资源池，你按需购买；云可以像自来水，电，煤气那样计费。

h.极其廉价

由于“云”的特殊容错措施可以采用极其廉价的节点来构成云，“云”的自动化集中式管理使大量企业无需负担日益高昂的数据中心管理成本，“云”的通用性使资源的利用率较之传统系统大幅提升，因此用户可以充分享受“云”的低成本优势，经常只要花费几百美元、几天时间就能完成以前需要数万美元、数月时间才能完成的任务。云计算可以彻底改变人们未来的生活，但同时也要重视环境问题，这样才能真正为人类进步做贡献，而不是简单的技术提升。

i.潜在的危险性

云计算服务除了提供计算服务外，还必然提供了存储服务。但是云计算服务当前垄断在私人机构（企业）手中，而他们仅仅能够提供商业信用。对于政府机构、商业机构（特别像银行这样持有敏感数据的商业机构）对于选择云计算服务应保持足够的警惕。

试用水印

云计算解决的问题：

- 普通用户硬盘可以用云。
- 互联网从业服务器可以用云。（阿里云，腾讯云，小鸟云）
- 云计算依靠网络
- 新生产环境

版本更新频率大增：推广运营也大大加强了力度，产品用户量增加，需求提高，更改 bug 多，更新频率高

熬夜加班：更新一般在半夜，尤其是大更新，如果出现问题的话，运维团队都不能离开，必须协助处理生产环境不稳定扯皮——对立：部门协调不稳定压力更大恶性循环

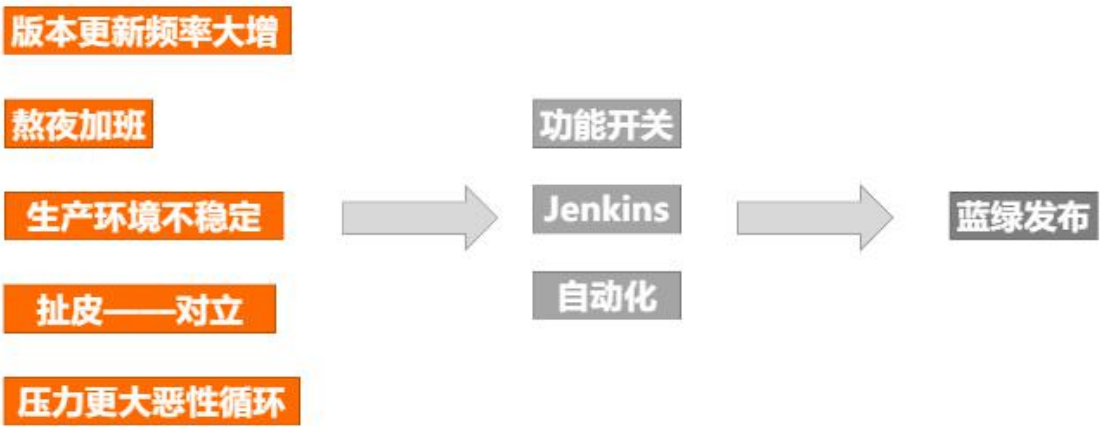
自动化：脚本自动化

Jenkins：Jenkins 是一个功能强大的应用程序，允许持续集成和持续交付项目，无论用的是什么平台。这是一个免费的源代码，可以处理任何类型的构建或持续集成。集成 Jenkins 可以用于一些测试和部署技术。Jenkins 是一种软件允许持续集成。

功能开关：为了快速发布开发完成的功能，现代的互联网企业通常会以比较快的迭代周期来持续的发布。但是有时候因为技术或者业务上的原因，需要在发布的时候将某些功能隐藏起来。一种解决方案是，在独立的分支上开发新功能，全部开发测试完成之后，才合并回主干，准备发布。这也就是我们经常提到的功能分支（feature branch）。本文将介绍如何使用功能开关（feature toggle）来更好地解决这个问题，及其在一个典型 Spring web 应用程序中的具体实现，最后讨论了功能开关和持续集成如何协同工作。

蓝绿发布：蓝绿部署是一种以可预测的方式发布应用的技术，目的是减少发布过程中服务停止的时间。简单来说，我们把整个服务集群分成两组（或更多组，为了显示蓝/绿，分为两组），一组认为是绿色，另一组认为是蓝色，此时通过 LB LIVE 都能够为用户提供服务。

试用水印



1、多点负载均衡单体应用。

2、混合云。

混合云提供既在公共空间又在私有空间中的服务。当公司需要使用既是公共云又是私有云的服务时，选择混合云比较合适。混合云把公用云模式与专用云模式结合在一起。混合云有助于提供所需的、外部供应的扩展。用公用云的资源扩充专用云的能力，可用来在发生工作负荷

二、运维生产环境的发展

devops

DevOps 是一个必然趋势，是一种方法，也是一种观念。

1：原始的互联网公司工作模式是瀑布流，但用户越多，需求越大，公司的管理，人力成本都是问题。而如果更新间隔太慢，一定会导致用户的满意程度下滑。DevOps 的观念应运而生。

2：DevOps 实际上是两个单词：Development 和 Operations。

3：开发人员：根据需求情况，把需求拆分成多个小需求，小步快跑大幅增加需求完成的频率。运维人员：运用自动化和 CI/CD 的概念，运用工具，实现稳定、快速的版本更新上线。

4：打破开发人员和运维人员的壁垒。

试用水印

5：对运维人员的技术要求和经验大大提升。

开发人员根据需求情况，把需求拆分成多个小需求，小步快跑大幅增加完成需求频率。运维人员是运用自动化 CI/CD 的概念，运用工具，实现稳定，快速的版本更新上线，运维和开发往底层说，实际就是岗位要的东西不一样，运维要的是保持业务的稳定持续运行，开发是为了快速开发，开发完就快速上线，一个要稳定，一个要赶紧上线，开发上线的越多，更新的就越多，运维就更不稳定

新环境产生问题：

问题 1：资源利用率。

服务器的治理，环境的迁移容灾上会产生问题，一个虚拟机要管多个配置的话，治理起来一定是难度更大的，端口容易冲突

一个虚拟机跑多个项目。治理、迁移、容灾?

问题 2：扩容不及时。

脚本化启动 50 台虚拟机要多久？脚本化启动 50 台容器要多久？环境部署要多久？风险大不大?

问题 3：环境不一致导致的问题。

多点负载均衡单体应用始终不变，然后又做成了 docker，开始用了 docker，第三个 devops 也没有变。

新型生产环境

生产环境：

多点负载均衡单体应用。

docker。

devops。



试用水印

多点负载均衡单体应用

- docker
- devops

三、docker 概念

概念:docker13 年初开源，公司本来叫 dotcloud，后改名叫 docker。被 Mitantis 收购。基于 Linux 内核的 cgroup，namespace，以及 AUFS 类的 Union FS 等技术，对进程进行封装隔离，属于操作系统层面的虚拟化技术。由于隔离的进程独立于宿主和其他的隔离的进程，因此也称其为容器。

四、docker 与容器区别

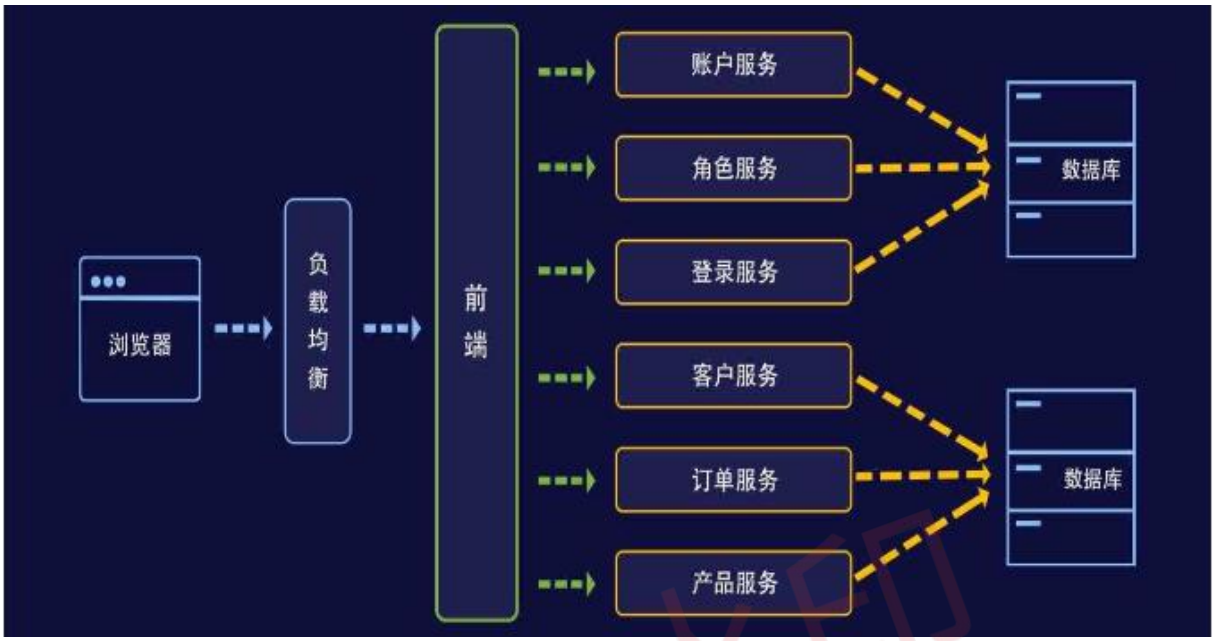
docker 就是容器，容器就是 docker，并不是这样的，容器就是一个技术类型。而 docker 是当下最主流的，容器的一种实现容器的方案，docker 只是容器其中一种实现方案，其他方案包括：LXC，Mesos，RKT 等等，最大区别当容器和服务器的数量达到一定规模的时候，就会碰到管理的问题，即如何有效管理大量的服务器和容器，保证应用的稳定运行、方便升级和故障的快速解决。容器编排工具提供图形化界面或者命令行来管理容器和服务器集群，提供容器配置、任务发布、服务发现、负载均衡、系统监控和故障恢复、声明式系统配置以及有关容器部署和性能的规则和约束定义机制等。

五、进程级封装概念

docker 或者容器和传统虚拟化最大的一点区别，就是虚拟化的封装是系统级的封装，docker 或者其他容器是进程级的封装。和传统虚拟化最大的一点区别，就是虚拟化是系统级的封装，进程级封装。用户量越来越大，功能越来越多。代码越来越重。耦合越来越高。一旦某个哪怕小的逻辑出错，甚至会影响整个站点。bug 频率大增，bug 频率增加耦合越来越高，简直对于互联网公司来说就是灾难。代码重复率高，维护难度大。代码越来越吃性能。服务器成本大增。

试用水印

六、微服务生产环境



微服务



- 混合云+docker+k8s。
- devops。

试用水印

七、k8s－容器编排管理工具

1.k8s 功能

微服务需要跑多个容器，容器多又会涉及到通信、架构、伸缩、更新、监控等等问题。

自愈：重新启动失败的容器，在节点不可用时，替换和重新调度节点上的容器。

弹性伸缩：通过监控容器的 cpu 的负载值，如果这个平均高于 80%，增加容器的数量，如果这个平均低于 10%，减少容器的数量。
服务的自动发现和负载均衡：不需要修改应用程序来使用不熟悉的服务发现机制。

2.k8s 作用

Kubernetes 为容器提供了自己的 IP 地址和一组容器的单个 DNS 名称，并可以在它们之间进行负载均衡。滚动升级和一键回滚： Kubernetes 逐渐部署对应用程序或其配置的更改，同时监视。

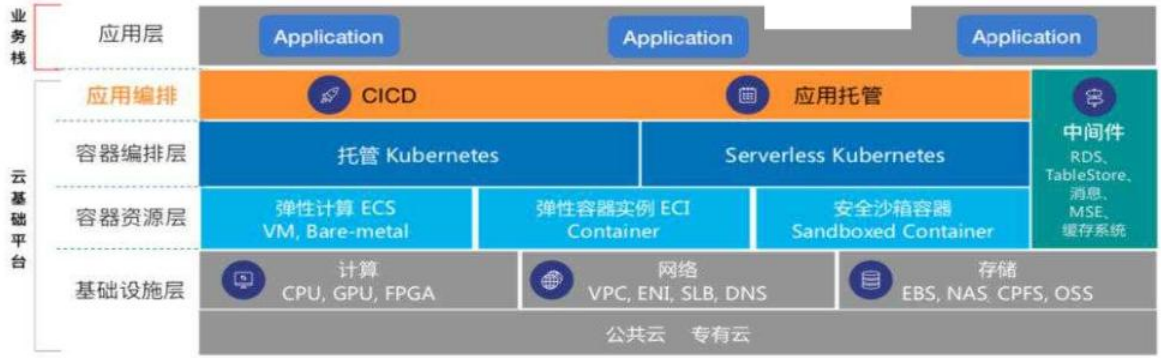
应用程序运行状况，以确保它不会同时终止所有实例。 如果出现问题，Kubernetes 会恢复更改，利用日益增长的部署解决方案的生态系统。

要使用 Kubernetes，你需要用 Kubernetes API 对象 来描述集群的 预期状态（desired state）：包括你需要运行的应用或者负载，它们使用的镜像、副本数，以及所需网络和磁盘资源等等。你可以使用命令行工具 kubectl 来调用 Kubernetes API 创建对象，通过所创建的这些对象来配置预期状态。你也可以直接调用 Kubernetes API 和集群进行交互，设置或者修改预期状态。一旦你设置了你所需的目标状态，Kubernetes 控制面（control plane） 会通过 Pod 生命周期事件生成器（ PLEG ），促成集群的当前状态符合其预期状态。为此，Kubernetes 会自动执行各类任务，比如运行或者重启容器、调整给定应用的副本数等等。Kubernetes 控制面由一组运行在集群上的进程组成。

试用水印

八、云原生技术栈的概念及技术

1.云原生技术栈



2.概念

- 云原生是一个生态概念、是一线互联网公司发展到某个极端的必然选择。
- 包含三大要素：容器及编排管理、DevOps、微服务。

3.技术模块

- 应用定义及部署（App Definition and Development）
- 编排与管理（Orchestration & Management）
- 运行环境（Runtime）
- 配置（Provisioning）
- 平台（Platform）
- 可观测性和分析（Observability and Analysis）
- 无服务（Serverless）

4.云原生时代

a：技术升级，概念先行。云原生、容器、devops 等一定是未来若干年的发展方向。

试用水印

b：为什么说容器划时代？一方面容器解决了大型架构的发展瓶颈，另一方面，从运维岗位讲，传统运维已经跟不上脚步了，会新技术的运维一定会取代传统运维，更别说其他运维岗位。

c：在我刚学运维时候有人和我说了一句话：运维这个岗位慢慢会消失。

d：docker 是云原生时代的基石也是应用基础。而这个基础也需要有一定的经验积累才能学通。

e：对运维的要求越来越高。

九、容器介绍

镜像（Image）和容器（Container）的关系，就像是面向对象程序设计中的 类 和 实例 一样，镜像是静态的定义，容器是镜像运行时的实体。

容器可以被创建、启动、停止、删除、暂停等 。

容器的实质是进程，但与直接在宿主执行的进程不同，容器进程运行于属于自己的独立的 命名空间。前面讲过镜像使用的是分层存储，容器也是如此。

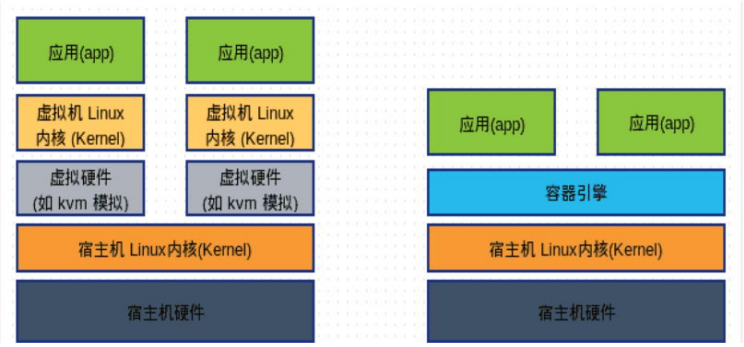
容器存储层的生存周期和容器一样，容器消亡时，容器存储层也随之消亡。
因此，任何保存于容器存储层的信息都会随容器删除而丢失。

十、容器和虚拟化介绍和比较

容器和虚拟化

1.介绍

试用水印



- 传统虚拟机，利用 hypervisor，模拟出独立的硬件和系统，在此之上创建应用。
- 虚拟机是一个主机模拟出多个主机。
- 虚拟机需要先拥有独立的系统。
- docker 是把应用及配套环境独立打包成一个单位。
- docker 是在主机系统中建立多个应用及配套环境。
- docker 是进程级的隔离。

2.容器和虚拟化比较

特性：

试用水印

阿里云 开发者训练营

容器和虚拟化

- 再次提高服务器资源利用率
- 重量更轻，体积更小。
- 匹配微服务的需求
- 保持多环境运行的一致性
- 快速部署迁移，容错高。

- 安全性较差
- 多容器管理存在难度。
- 稳定性较差
- 排错难度较大

- 启动速度 容器一〉秒级 虚拟机一〉分钟级
- 性能 容器一〉接近原生 虚拟机一〉较弱
- 内存代价 容器一〉很少 虚拟机一〉较多
- 硬盘使用 容器一〉一般为 MB 虚拟机一〉一般为 GB
- 运行密度 容器一〉单机支持千个容器 虚拟机一〉一般几十个
- 隔离性 容器一〉安全隔离 虚拟机一〉完全隔离
- 迁移性 容器一〉优秀 虚拟机一〉一般
- 容器：再次提高服务器资源利用率
- 虚拟机：安全性较差
- 容器：重量更轻，体积更小
- 虚拟机：多容器管理存在难度
- 容器：匹配微服务的需求
- 虚拟机：稳定性较差
- 容器：保持多环境运行的一致性
- 容器：快速部署迁移，容错高
- 虚拟机：排错难度较大

试用水印

3.容器与虚拟化的区别：

阿里云 开发者训练营

容器和虚拟化

镜像可以看做是一个压缩包，里面包含所需要的应用和应用要的配置文件及底层的库、参数、环境变量等。
容器是运行这个压缩包，实现具体功能的存在。

我们来思考以下三个问题：

Q1：我们运行应用，底层总是少不了系统环境的，而每个镜像都需要用，这样不会占用大量存储吗？
不会，docker可以将一个基础系统镜像可以被多个镜像共用。这里可以代入调用和缓存的概念。保证每个容器体积小，速度快，性能优。

Q2：我们通过镜像启动容器去使用，更新文件是必然的事情，那镜像岂不是会被多个容器更改，造成冲突？
镜像采用了分层设计，启动容器后，镜像永远是只读属性，只不过在最上层加一层读写层（容器层），如果需要底层镜像的文件进行更改，读写层会复制一份镜像中的只读层进行写操作，这就是Copy-On-Write。

Q3：我们想要在一个镜像（Centos7）上创建一个新镜像（Centos7+nginx），新镜像会复制底层镜像吗？
不会，参考问题1。

Docker 底层技术：

Namespaces：

作用：访问隔离

Docker 主要就是借助 Linux 内核技术 Namespace 来做到隔离的，Linux Namespaces 机制提供一种资源隔离方案。PID,IPC,Network 等系统资源不再是全局性的，而是属于某个特定的 Namespace。每个 namespace 下的资源对于其他 namespace 下的资源都是透明，不可见的。因此在操作系统层面上看，就会出现多个相同 pid 的进程。系统中可以同时存在两个进程号为 0,1,2 的进程，由于属于不同的 namespace，所以它们之间并不冲突。而在用户层面上只能看到属于用户自己 namespace 下的资源，例如使用 ps 命令只能列出自己 namespace 下的进程。这样每个 namespace 看上去就像一个单独的 Linux 系统。

IPC\NetWork\mount\PID\UTS\User

Control groups：

作用：做资源控制，CPU\MEM\宽带等。

提供的一种可以限制、记录、隔离进程组所使用的物理资源（如 cpu、memory、磁盘 IO 等等）的机制，被 LXC、docker 等很多项目用于实现进程资源控制。

cgroup 将任意进程进行分组化管理的 Linux 内核功能。

cgroup 本身是提供将进程进行分组化管理的功能和接口的基础结构，I/O 或内存的分配控制等具体的资源管理功能是通过这个功能来实现的。

Rootfs:

作用：文件系统隔离

访问隔离案例

a.挂载镜像文件 iso

```
[root@docker ~] mount -t iso9660 zechen.iso /mnt
[root@docker ~] ls /mnt
```

试用水印

b.管理服务器

#创建命名空间

[root@docker ~] unshare -m /bin/bash

#重新挂载

[root@docker ~] mount -t iso9660 -0 loop zechen.iso /mnt

#重新查看

[root@docker ~] ls /mnt

自己能看到，切换到其他用户就不能看见文件

退出访问隔离

[root@docker ~] exit

Docker 版本：docker 官方提供 2 种版本，一个是 docker 企业版 docker-EE，另外一个则是社区版 docker-ce，我们在学习或者测试环境使用 docker-ce 版本即可。发布版本 docker 更新很快，现在最新的是：20.10 进入官网查看版本发行时间和版本号。

试用水印

十一、答疑环节

1.docker 容器怎么迁移

docker 容器迁移，前期持久化数据，固定的目录，数据然后映射到这个目录上面永久存储到某一个目录上面，导致的一些配置出现问题。还有要注意的就是 docker 的版本。

2.关于云原生的技术栈

云原生的技术栈实际上三个方面，devops，微服务，还有一块是容器 K8s，从系统层次来看，从上到下分别是：

- **应用层：**应用定义及部署（App Definition and Development)、配置（Provisioning)、可观测性和分析（Observability and Analysis)、无服务（Serverless)
- **集群：**编排与管理（Orchestration & Management)
- **底层运行环境：**运行环境（Runtime)

3.数据库应该安装 docker 里面

建议数据库不要装 docker，放到你的虚拟机里边，数据库本身这个软件也不太可能和其他的软件共同装在一个，毕竟容器里你进去以后你要排错，为了避免这种麻烦，建议数据库这一块就尽量不要放在容器里边。

4.启动 docker 用什么脚本

建议学一下 k8s，自己的脚本也可以运行。

docker 镜像是简化的 linux

镜像里面只包含两种东西，它的依赖环境和它实际要做的一个应用，镜像是把它两个融合在一起的，就是应用和它底层的一些依赖环境变量，或者一些参数也好等等。它把它们融合到一起了。

5.容器是什么



一艘船的载重比如是 10000 吨，卡车本身体积很大，所有卡车上船加上货物也就 2000 吨，大部分载重都浪费了。但船不能少，油费人工等成本也不能少。这怎么办？是不是可以造小一点的船，载重也不用那么大。多造几艘。这样是不是解决了？后来仔细一想，卡车完全没必要上船，只要货物上去就可以了！到了码头再卸下来。

试用水印

有人专门做过实验：如果运啤酒，用原始的卡车方法全程陆运，每吨成本是 27 美金。且非常慢。如果用卡车运送到码头——卸载到船上——船到另一个码头——再装再到终端卸载，成本是 25 美分！节省了 99%左右的运输成本。且因为船的负载和卡车负载根本没法比，效率还大大提高了。之后 Container（集装箱，容器）改变了整个世界物流行业的面貌。极大地推动了物流生产力的发展。到现在，集装箱都是外贸、零售、电商等交易的必不可少的环节。

我个人不太喜欢把 container 翻译成容器，我更想把他叫做“集装箱”。现在是谁把“容器”命名成为 container 我暂时没办法考证，但我真心佩服这位命名者。它起这个名字，首先是非常形象，甚至解决的问题和解决的方案都能看到集装箱的影子。更重要的是，我能感受到他想要改变 IT 生态的一个理想和决心，而且他做到了！如他的命名。容器是什么？简单说，他就是一个集装箱，这个集装箱可以在各个系统之间来回搬运，集装箱里面，放着的是一个或多个应用，以及运行环境。作用简而言之，就是把环境和应用封装成一个标准的单元，用于开发、部署、交付等环节。

他解决了几个重要的问题：

- 一个应用，因为环境不同，可能会出现各种各样本可以避免的问题。
- 一个系统中，多个应用之间可能引发的各种冲突故障。
- 相比传统虚拟化，不吃资源、节省成本。

试用水印

容器、镜像和仓库

演讲人：马哥教育李老师 阿里云 IoT 技术专家

视频链接：https://developer.aliyun.com/learning/course/892



一、Docker 底层技术概述



Namespaces：

作用：访问隔离

Docker 主要就是借助 Linux 内核技术 Namespace 来做到隔离的，Linux Namespaces 机制提供一种资源隔离方案。PID,IPC,Network 等系统资源不再是全局性的，而是属于某个特定的 Namespace。每个 namespace 下的资源对于其他 namespace 下的资源都是透明，不可见的。因此在操作系统层面上看，就会出现多个相同 pid 的进程。系统中可以同时存在两个进程号为 0,1,2 的进程，由于属于不同的 namespace，所以它们之间并不冲突。而在用户层面上只能看到属于用户自己 namespace 下的资源，例如使用 ps 命令只能列出自己 namespace 下的进程。这样每个 namespace 看上去就像一个单独的 Linux 系统。

IPC\NetWork\mount\PID\UTS\User

Control groups：

作用：做资源控制，CPU\MEM\宽带等

提供的一种可以限制、记录、隔离进程组所使用的物理资源(如 cpu、memory、磁盘 IO 等等)的机制，被 LXC、docker 等很多项目用于实现进程资源控制。cgroup 将任意进程进行分组化管理的 Linux 内核功能。cgroup 本身是提供将进程进行分组化管理的功能和接口的基础结构，I/O 或内存的分配控制等具体的资源管理功能是通过这个功能来实现的。

Rootfs:

作用：文件系统隔离

二、访问隔离案例

挂载镜像文件 iso

```
[root@docker ~] mount -t iso9660 zechen.iso /mnt
[root@docker ~] ls /mnt
```

管理服务器

#创建命名空间

```
[root@docker ~] unshare -m /bin/bash
```

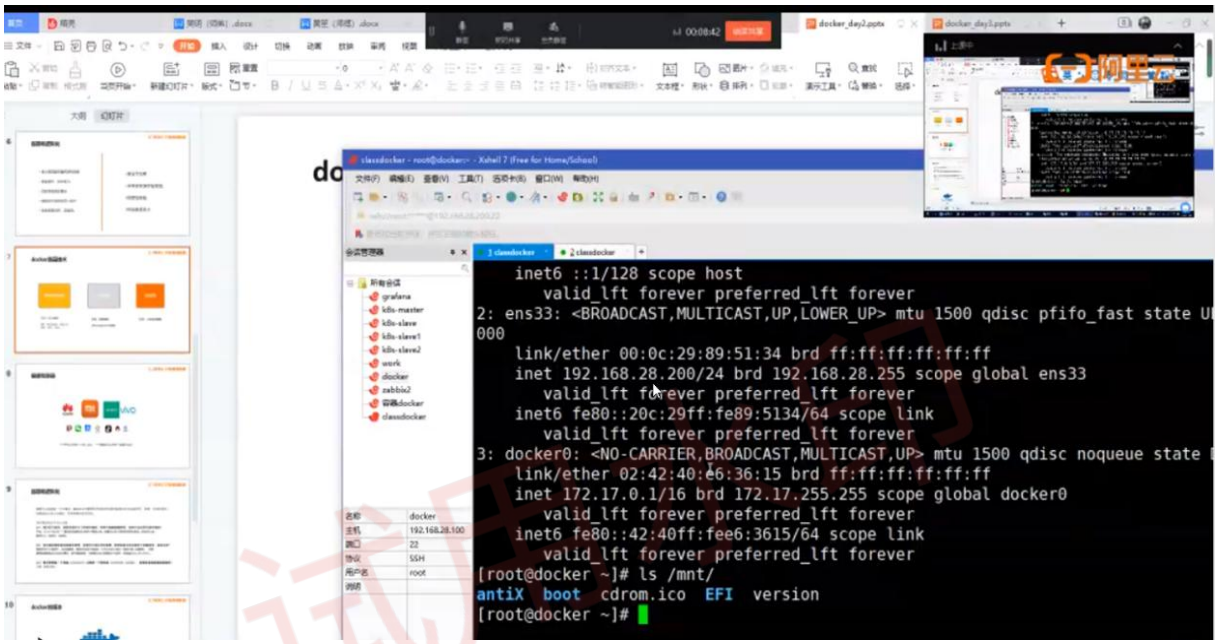
#重新挂载

[root@docker ~] mount -t iso9660 -0 loop zechen.iso /mnt

#重新查看

[root@docker ~] ls /mnt

自己能看到，切换到其他用户就不能看见文件



这个案例让大家理解一下，感受一下命名空间大致内容。联想到做出隔离的作用。每个容器之间都是隔离的。mount 其他的东西都隔离的。

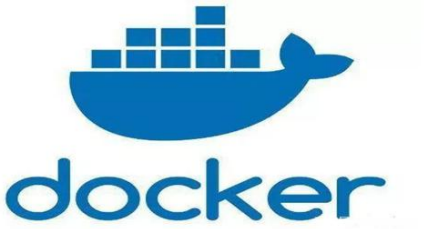
三、Docker 版本介绍

docker 官方提供 2 种版本，一个是 docker 企业版 docker-EE，另外一个则是社区版 docker-ce,我们在学习或者测试环境使用 docker-ce 版本即可。发布版本 docker 更新很快，现在最新的是:20.10

进入官网查看版本发行时间和版本号：

阿里云 开发者训练营

docker的版本



docker官方提供2种版本，一个是docker企业版docker-EE，另外一个则是社区版docker-ce，
我们在学习或者测试环境使用docker-ce版本即可。

发布版本docker更新很快，现在最新的是：

四、Docker 安装部署

docker 常见的有 3 种安装方式，yum、rpm 包、脚本，我们采用相对简单但对各种环境比较友好的方式:(关防火墙和 selinux)。

#安装存储库拓展包 yum install-y yum-utils
#设定存储库 yum-config-manager\ --add-repo\
htts://download.docker.com/linux/centos/docker-ce.repo
#安装三个包
yum-y install docker-ce docker-ce-cli containerd.io
#验证
docker version#查询 docker 版本信息
docker info#查询 docker 详细信息

试用水印



在安装之前一定要检查一下 firewood，并且确定环境用的这个镜像系统镜像都是老师给的。

首先我们要安装拓展的一个存储库，第一可以拓展，第二个稳定。

第二个命令，安装。第一个包 docker 的程序包。第二个是 docker 的一个工具包。第三个 container.io 是做系统和这个 docker 的一个 api 的一个守护进程。

第三，安装这三个包，检查版本，查询信息。如果有正确的打印或者正确的输出，说明安装成功。

五、docker 安装与启动

yum install -y epel-release
yum install docker-io # 安装 docker
配置文件 /etc/sysconfig/docker
chkconfig docker on # 加入开机启动
service docker start # 启动 docker 服务
基本信息查看
docker version # 查看 docker 的版本号，包括客户端、服务端、依赖的 Go 等

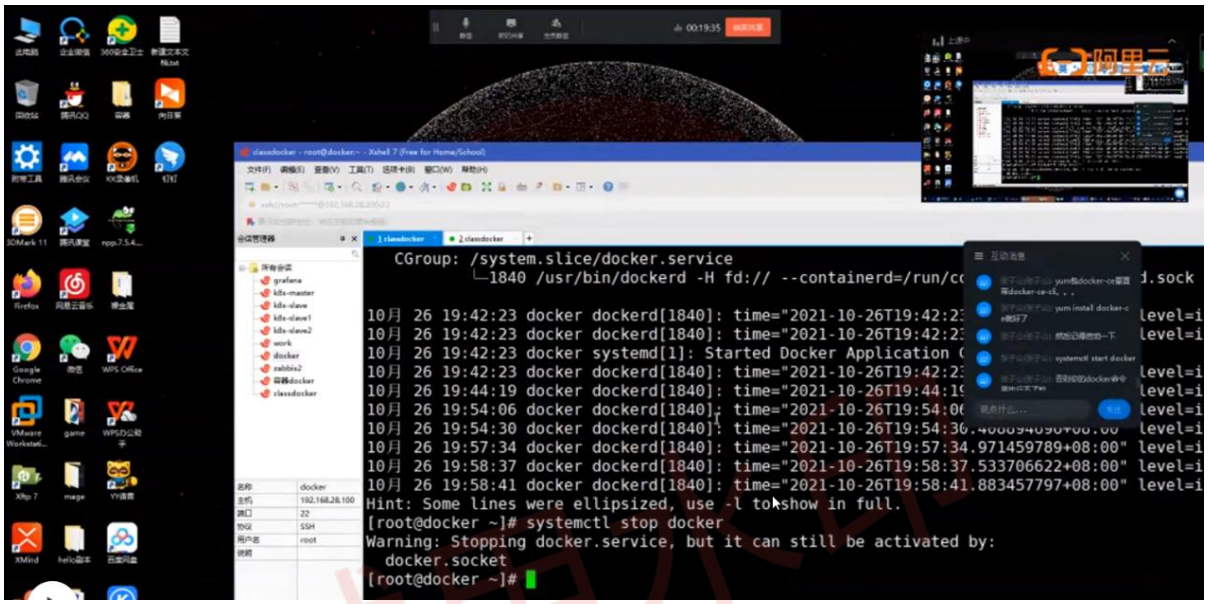
试用水印

docker info # 查看系统(docker)层面信息，包括管理的 images, containers 数等

docker pull centos 下载

docker images [centos] 查看

docker run -i t centos /bin/bash



六、Docker 常用命令解释

阿里云 开发者训练营

docker常用命令

我们知道了镜像和容器分别是什么，他们有什么用，那镜像哪来的呢？
第一种方式：官方镜像仓库。

#查询本地镜像
docker images

#查找镜像
docker search [images]

#下载镜像
docker pull [images:tag]

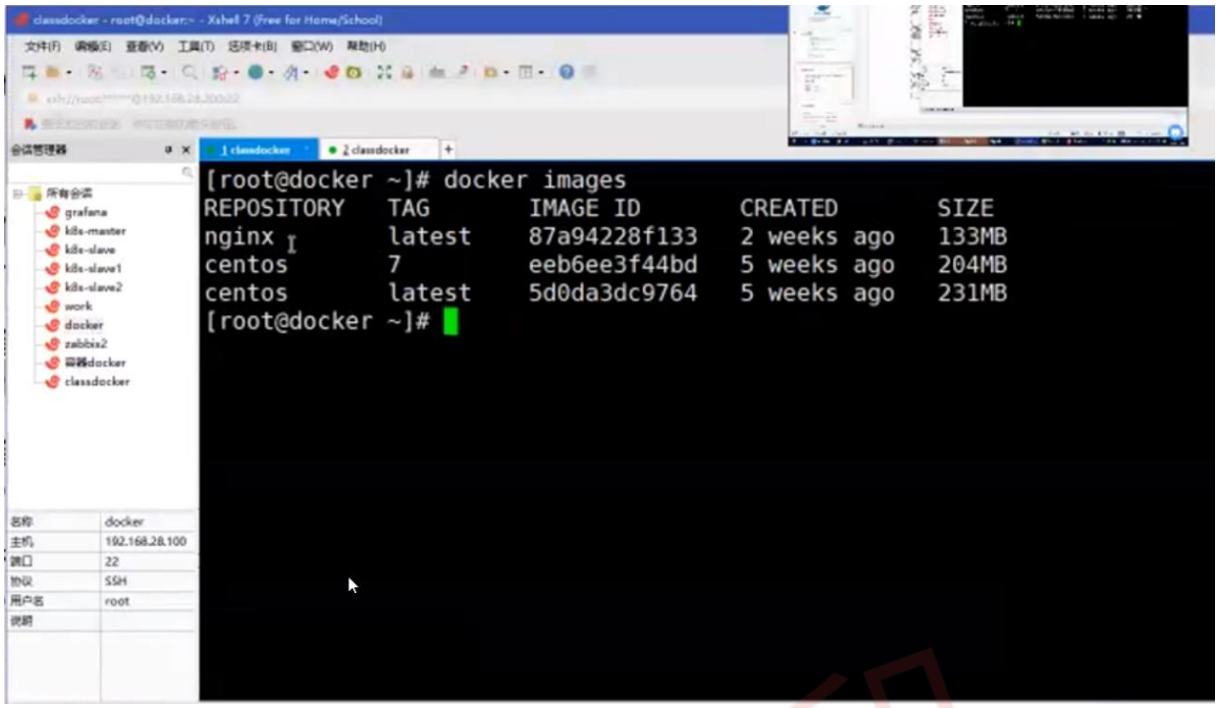
docker 常用命令

启动的 docker，镜像和容器分别是什么。可以理解为这个镜像。就是 u 盘。u 盘可以在很多的电脑上面，无论是笔记本还是台式或者一体机，甚至有一些手机，可以把接口转换一下，然后形成 usb 接口，都是可以使用这个 u 盘。这个 u 盘就是镜像；这个容器，就可以理解为是电脑。

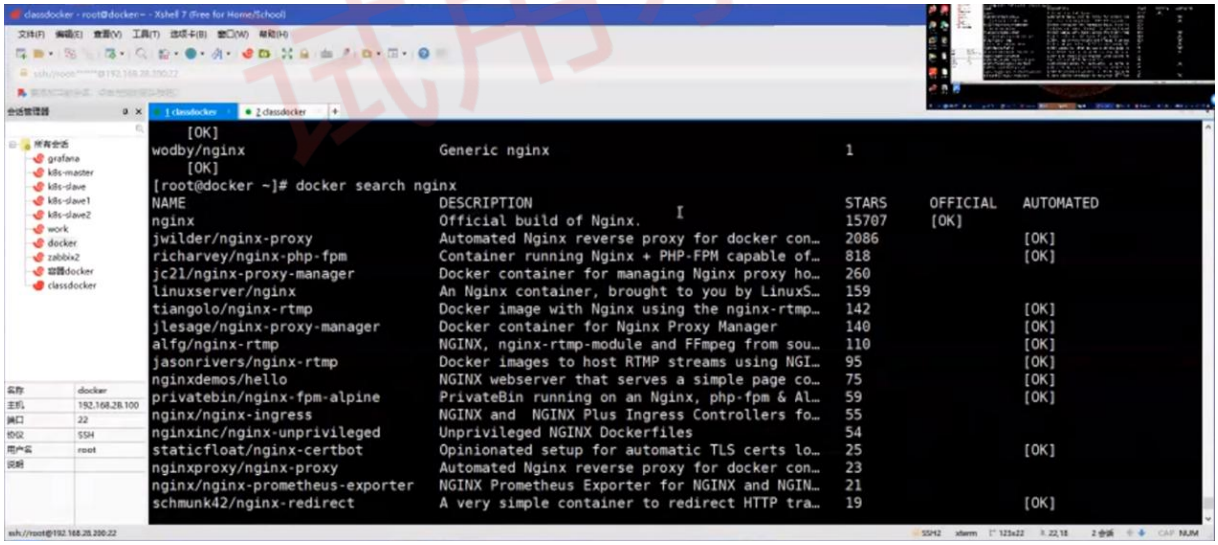
镜像来源。一般情况主流的方法是两种。一种是官方镜像仓库另一种是企业级的一个镜像仓库，把企业里边要用的都放到 harbor 里边。第一种方式:官方镜像仓库。

#查询本地镜像
docker images
#查找镜像
docker search[images]
#下载镜像
docker pull[images:tag]
镜像的获取
搜索镜像
docker search <image> # 在 docker index 中搜索 image
下载镜像
docker pull <image> # 从 docker registry server 中下拉 image
查看镜像
docker images： # 列出 images
docker images -a # 列出所有的 images（包含历史）
docker rmi <image ID>： # 删除一个或多个 image

试用水印



找一下官方镜像仓库的镜像，然后下载下来。



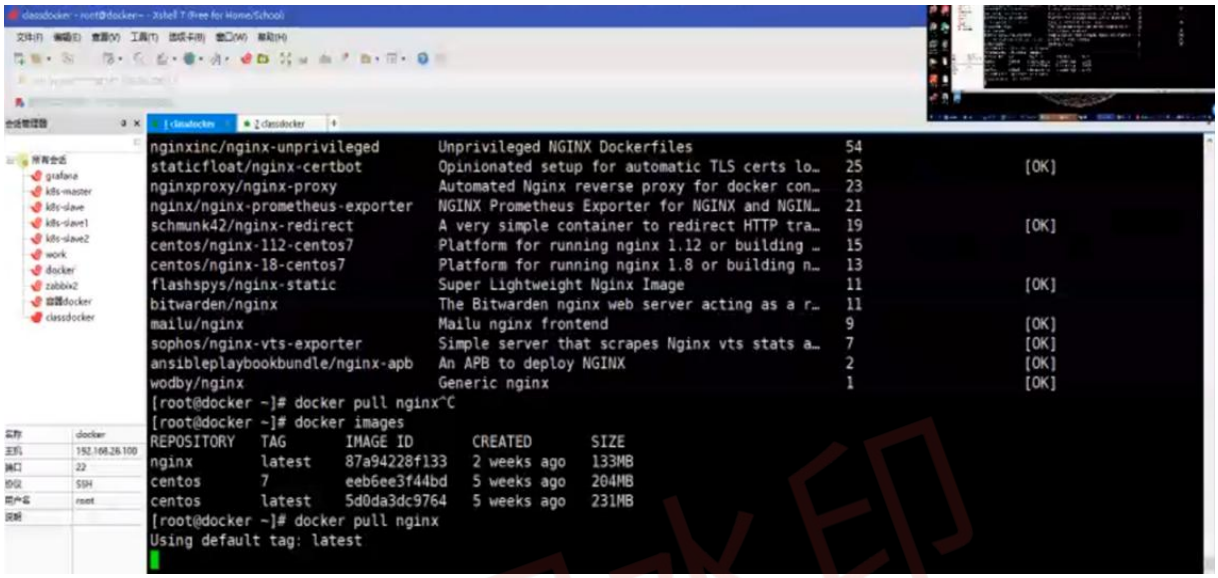
一共有五列，有两列是一定要注意的，强调一下第一列名字，就是你要找的这个名字。，实际上都列出来了。筛选出来之后，还要去注意一个地方 office，这下边儿有一个（OK）是官方提供的这个名字的镜像。

第一个查看一下你自己现在本地已有的镜像。

试用水印

第二步查找一下你的镜像，并且找到那个官方提供的版本的镜像。

第三步下载下来。



容器的使用

使用镜像创建容器

docker run -i -t sauloal/ubuntu14.04

docker run -i -t sauloal/ubuntu14.04 /bin/bash # 创建一个容器，让其中运行 bash 应用，退出后容器关闭

docker run -itd --name centos_aways --restart=always centos

#创建一个名称 centos_aways 的容器，自动重启

--restart 参数：always 始终重启；on-failure 退出状态非 0 时重启；默认为，no 不重启

查看容器

docker ps ：列出当前所有正在运行的 container

docker ps -l ：列出最近一次启动的 container

docker ps -a ：列出所有的 container（包含历史，即运行过的 container）

docker ps -q ：列出最近一次运行的 container ID

试用水印

再次启动容器

docker start/stop/restart <container> #：开启/停止/重启 container

docker start [container_id] #：再次运行某个 container （包括历史 container）

#进入正在运行的 docker 容器

docker exec -it [container_id] /bin/bash

docker run -i -t -p <host_port:contain_port> #：映射 HOST 端口到容器，方便外部访问容器内服务，host_port 可以省略，省略表示把 container_port 映射到一个动态端口。

删除容器

docker rm <container...> #：删除一个或多个 container

docker rm `docker ps -aq` #：删除所有的 container

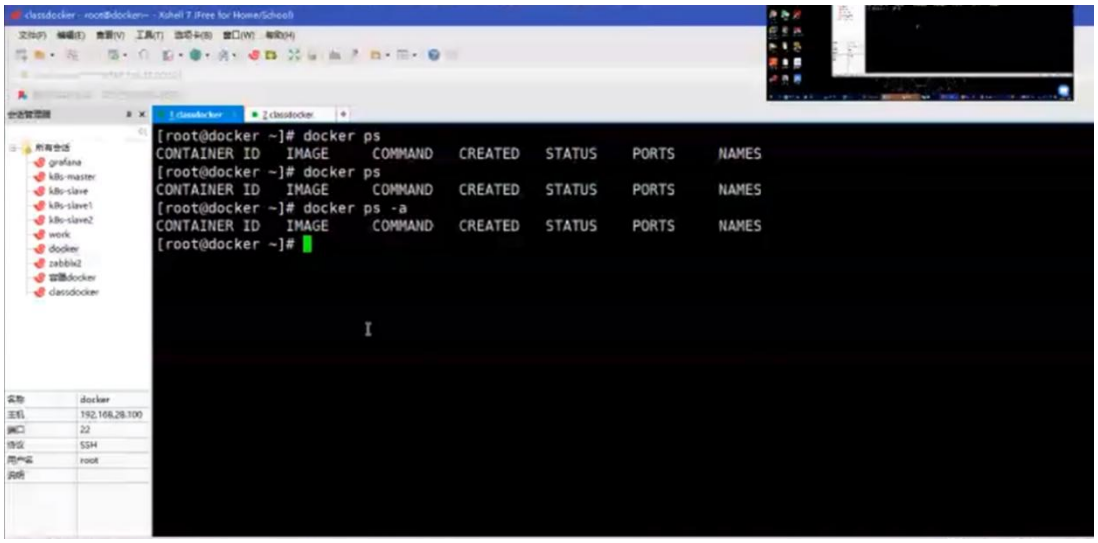
docker ps -aq | xargs docker rm #：同上，删除所有的 container

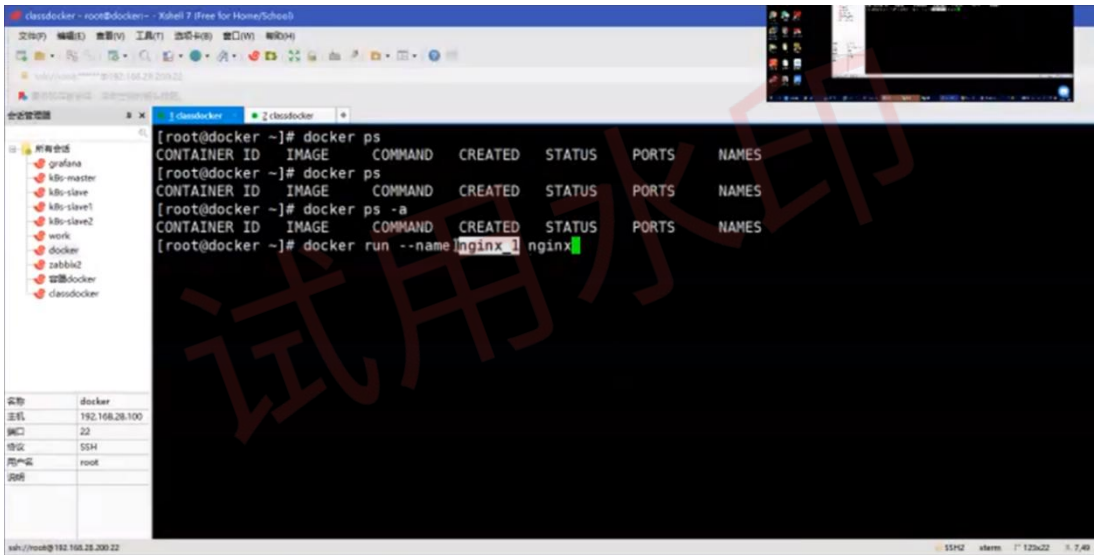
创建容器基础的命令格式。

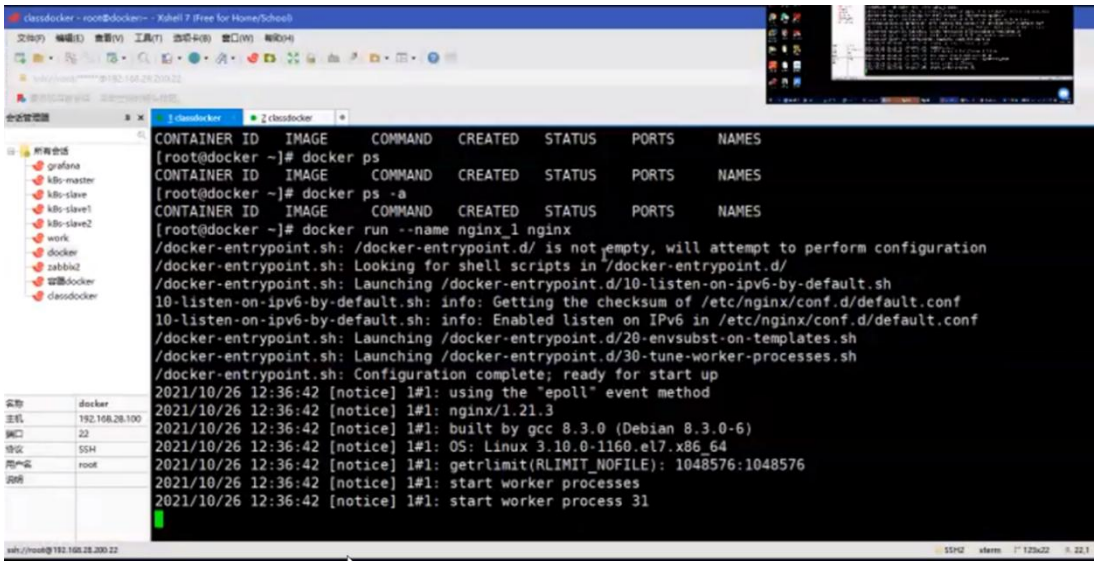


利用镜像直接创建容器:

docker run+参数[images:tag] 启动命令

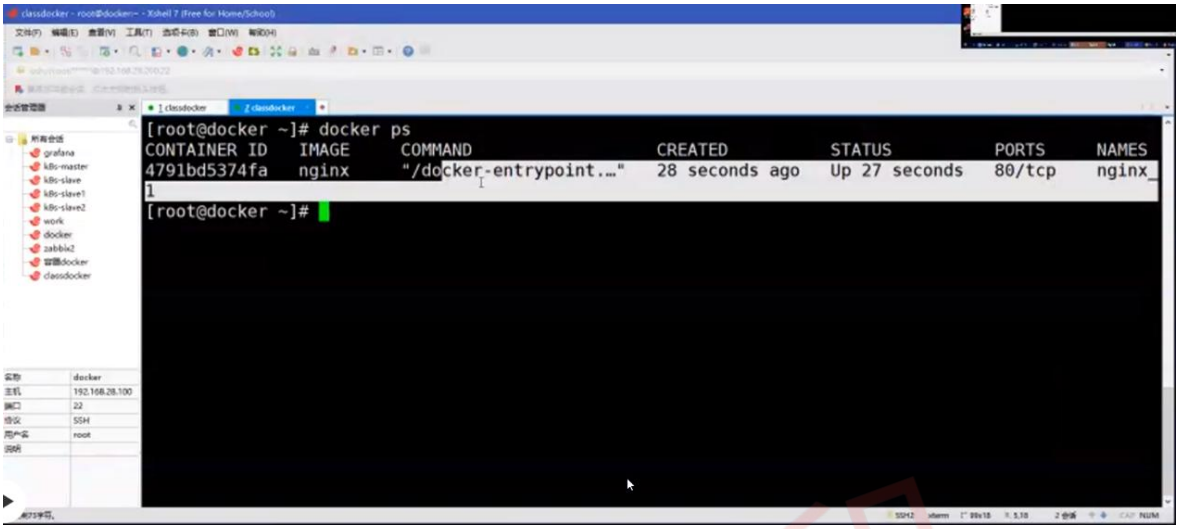

[root@docker ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]#

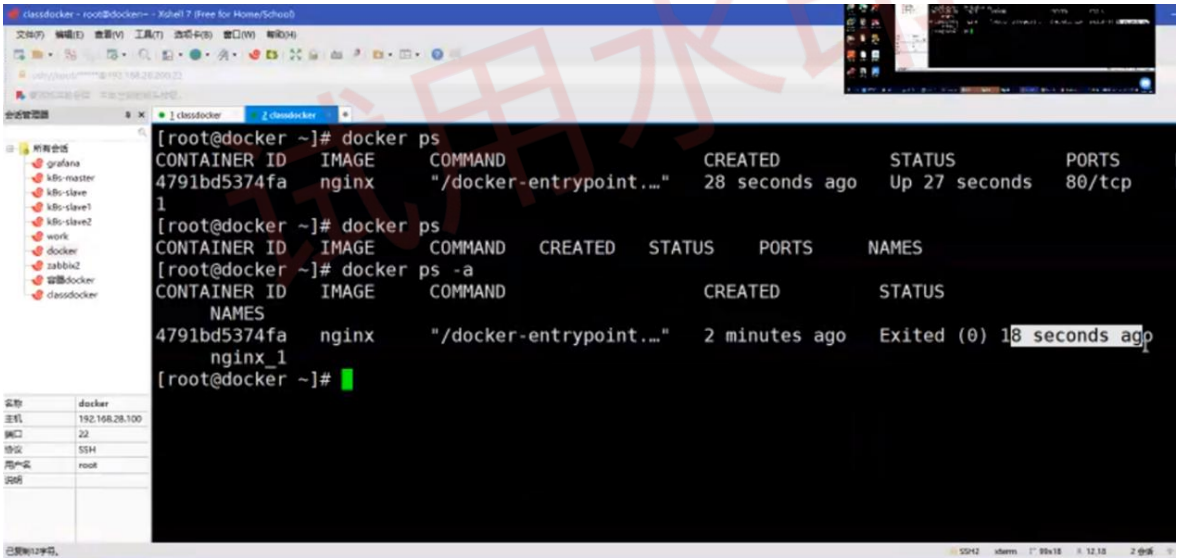

[root@docker ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker run --name nginx_1 nginx


CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@docker ~]# docker run --name nginx_1 nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2021/10/26 12:36:42 [notice] 1#1: using the "epoll" event method
2021/10/26 12:36:42 [notice] 1#1: nginx/1.21.3
2021/10/26 12:36:42 [notice] 1#1: built by gcc 8.3.0 (Debian 8.3.0-6)
2021/10/26 12:36:42 [notice] 1#1: OS: Linux 3.10.0-1160.el7.x86_64
2021/10/26 12:36:42 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2021/10/26 12:36:42 [notice] 1#1: start worker processes
2021/10/26 12:36:42 [notice] 1#1: start worker process 31

#查看现有容器

docker ps -a(加了该选项可以查出未启动的容器)



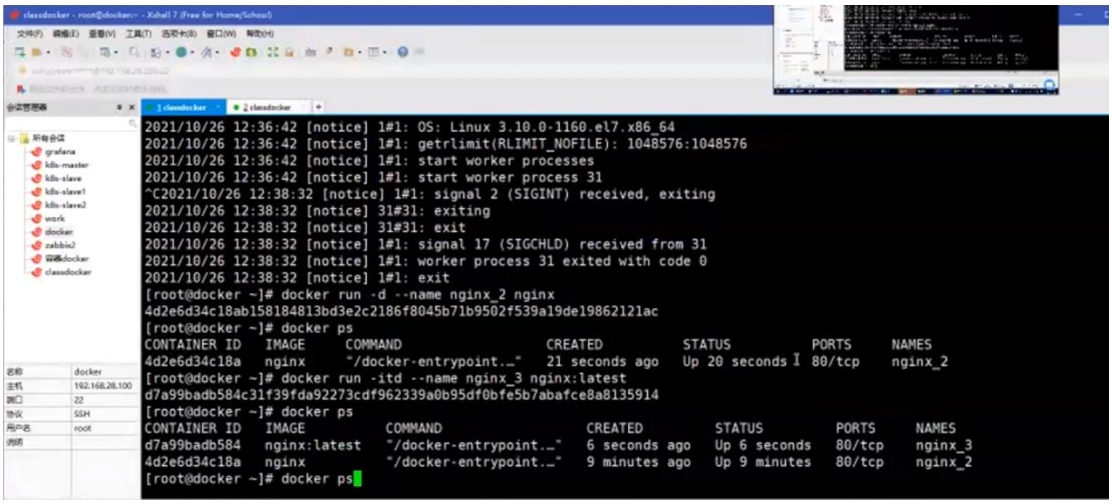


#指定容器名字

docker run --name[name][image:tag]

#利用镜像直接创建容器

docker run -d --name nginx_1 nginx:latest



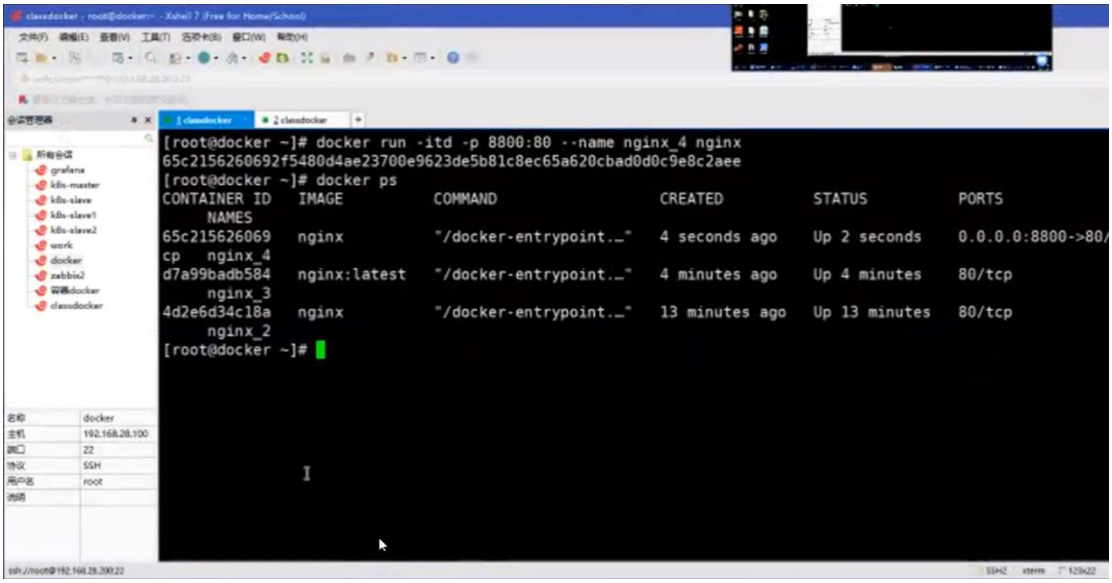
#镜像用可交互的方式创建容器

docker run-id--name nginx_1 nginx:latestt 以交互模式运行容量 t 为容器重新分配一个为输入终端

#创建容器并暴露端口

docker run-itd-p 8800:80--name nginx_1 nginx:latest

前面的是宿主机，就是操作 docker 的这个主机，你要暴露的端口。如果有用户想要访问你这个虚拟机里面的容器，或者你这个宿主机的容器，只需要访问 8800 这个端口就可以。用户访问的都是访问的宿主机的这个端口网络进来的，连接再通过内网的形式找到容器里面的 80 端口来进行实际的访问。



试用水印

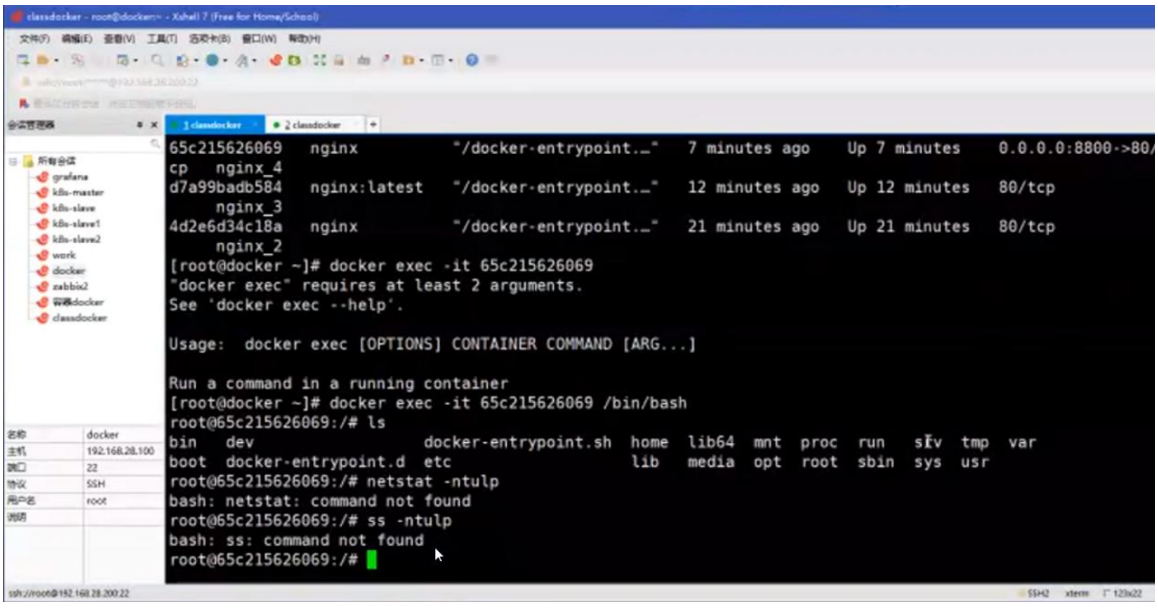
验证



#进入容器

docker exec -it[容器 ID]命令

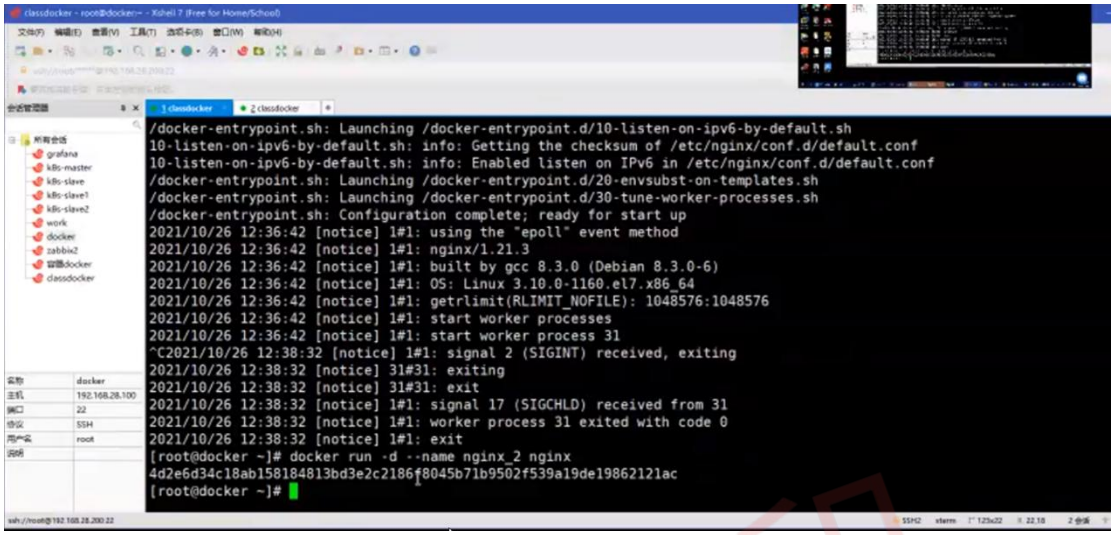
-it 不是一条命令，是两个选项。首先-it 是以交互模式运行容器，交互模式运行容器，-t 为容器，重新分配一个伪终端输入。分配一个伪终端，并且以交互的模式运行。-d 启动不成功，尝试用-itd 去启动



试用水印

#如何启动和停止容器

docker start/stop [容器 ID]



#如何删除已停止的容器

docker rm[容器 ID]

#如何删除所有已停止的容器

docker rm'docker ps-aq'

#停止所有的 container，这样才能够删除其中的

images:docker stop $(docker ps -aq)

#如果想要删除所有 container 的话再加一个指令：

docker rm $(docker ps -aq)

#查看当前有些什么 images

docker images

#删除 images，通过 image 的 id 来指定删除谁 docker rmi <image id>

#想要删除 untagged images，也就是那些 id 为<None>的 image 的话可以用

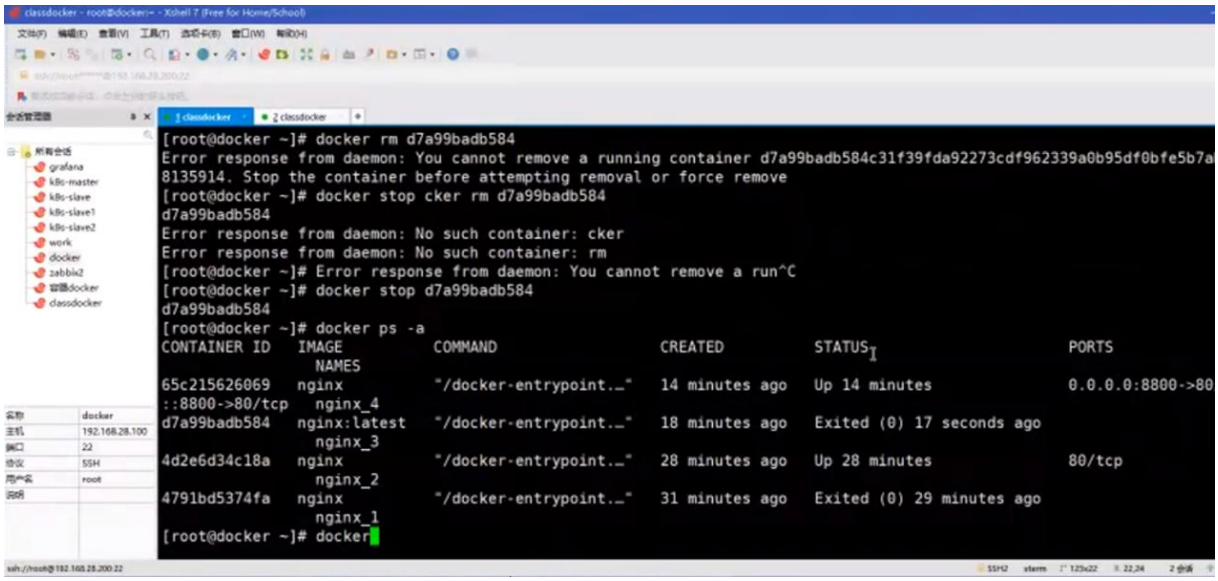
docker rmi $(docker images | grep "^<none>" | awk "{print $3}")

#要删除全部 image 的话 docker rmi $(docker images -q)

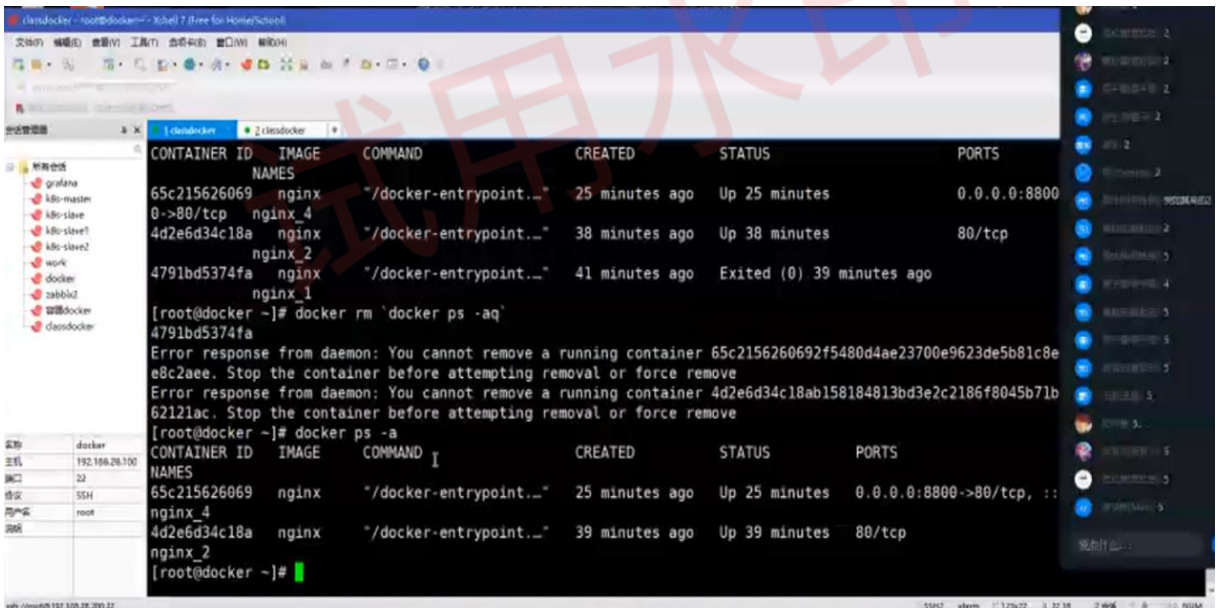
#docker rmi -f tomcat nginx centos：强制删除多个本地镜像

#docker rmi -f $(docker images -q)：类似于 linux 的管道，删除$表达式

试用水印



删除已停止的容器。



七、docker 基础命令

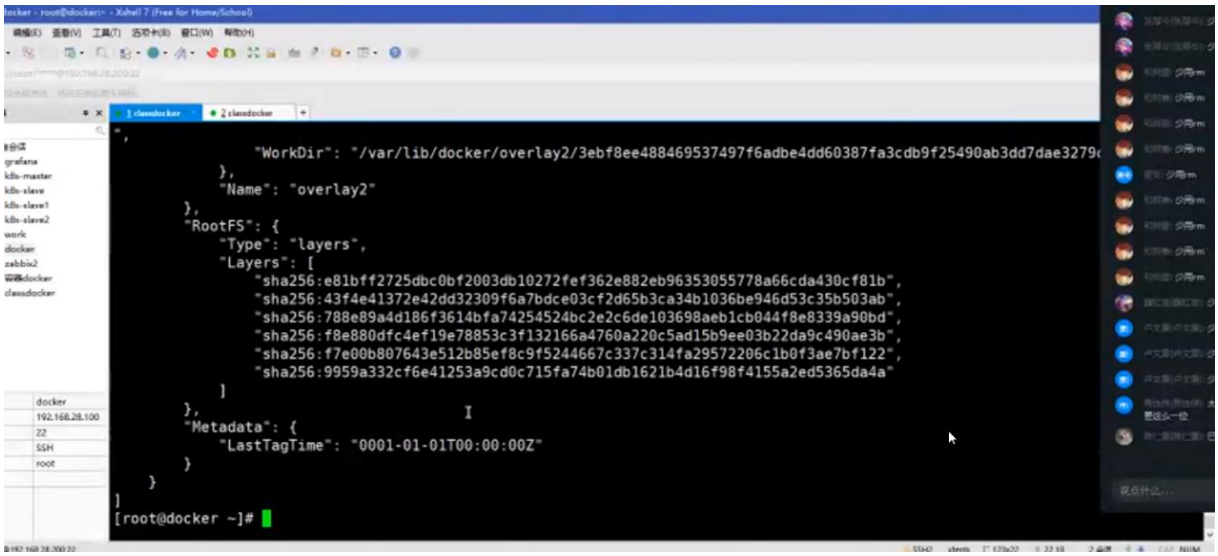


#查看镜像或容器的详细信息:

docker inspect[容器 ID/镜像名:tag]

#给镜像添加一一个软链接并改名和标签:

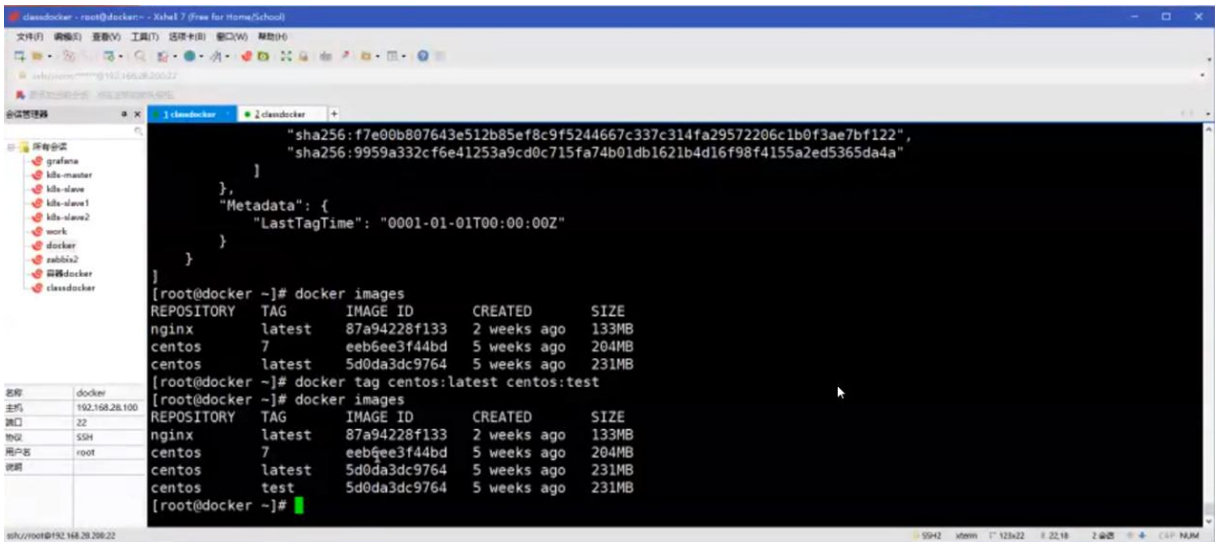
docker tag[oldname:tag][newname:tag]



#删除镜像:

docker rmi 镜像:tag/镜像 ID

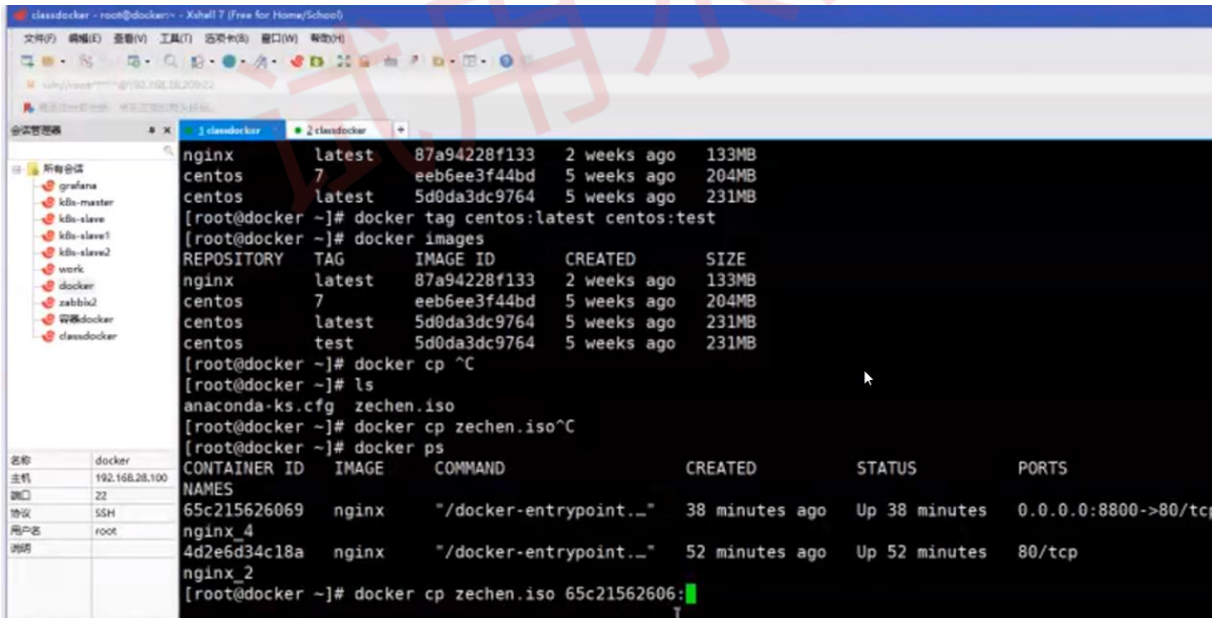
试用水印



#容器和宿主机之间文件复制

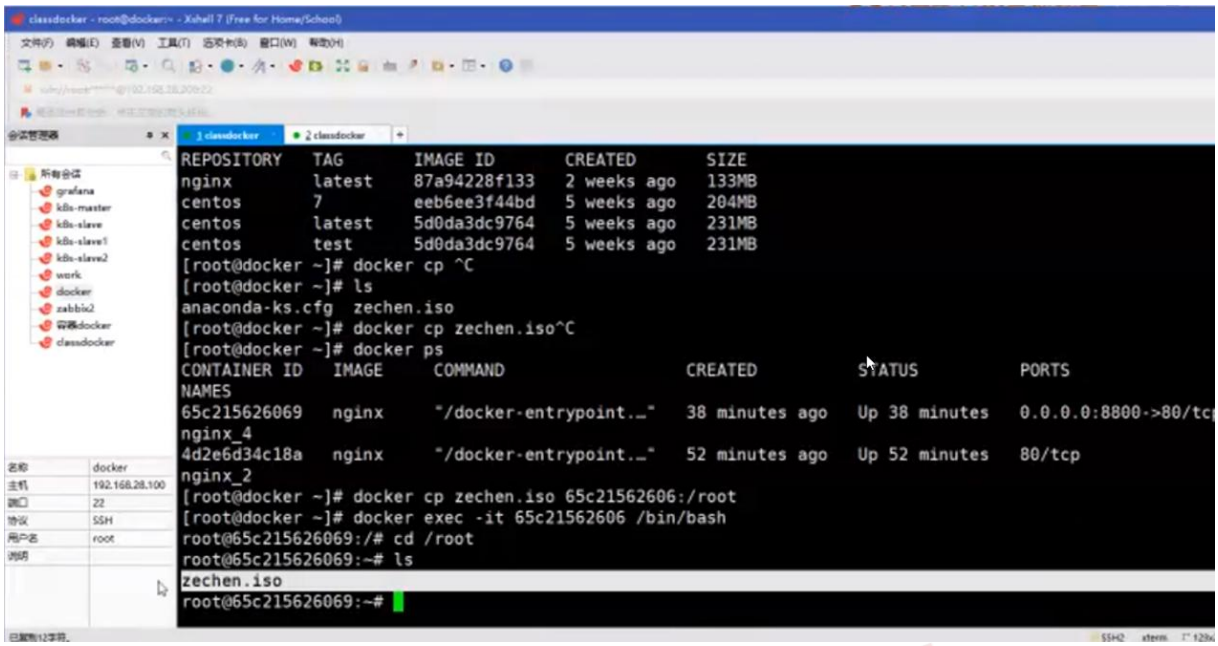
docker cp [文件目录容器 ID:内部路径]

docker cp [容器 ID:内部路径文件目录]

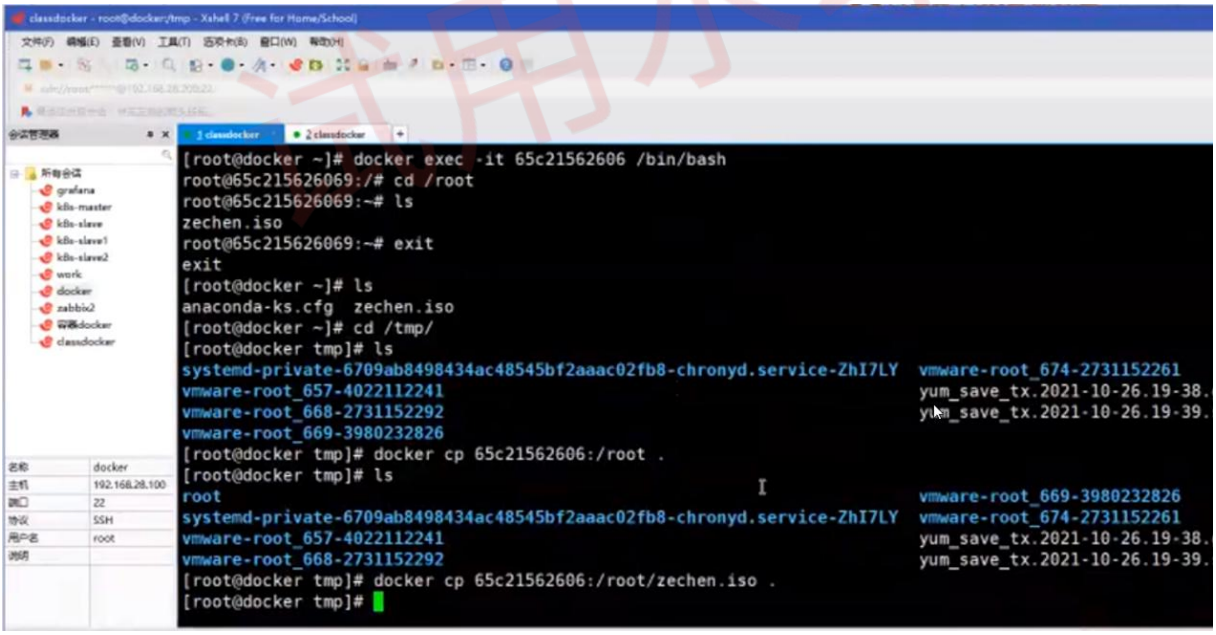


验证

试用水印



把刚刚这个容器里边的 iso 的这个镜像文件放到宿主机的根下的 tmp 下。



docker 的实际运用

演讲人：马哥教育李老师 阿里云 IoT 技术专家

视频链接：https://developer.aliyun.com/learning/course/892







一、docker 基础命令



试用水印

从天池基础镜像中获取

pythondocker pull 天池镜像：标签；

docker 构建镜像

docker?build?-t?registry.cn-shanghai.aliyuncs.com/target:myversion

docker 构建镜像并指定 DockerFile

docker?build?-f?./dockerfile?-t?registry.cn-shanghai.aliyuncs.com/target:myversion

设置默认工作目录

add . /目录名称

镜像启动后-----执行

sh run.sh,我们目前使用 cmd [“sh”,“run.sh”]

查看镜像或容器的详细信息:

docker inspect [容器 ID/镜像名:tag]

给镜像添加一个软链接并改名和标签:

docker tag [oldname:tag]

[newname:tag]

删除镜像:

docker rmi 镜像:tag/镜像 ID

容器和宿主机之间文件复制

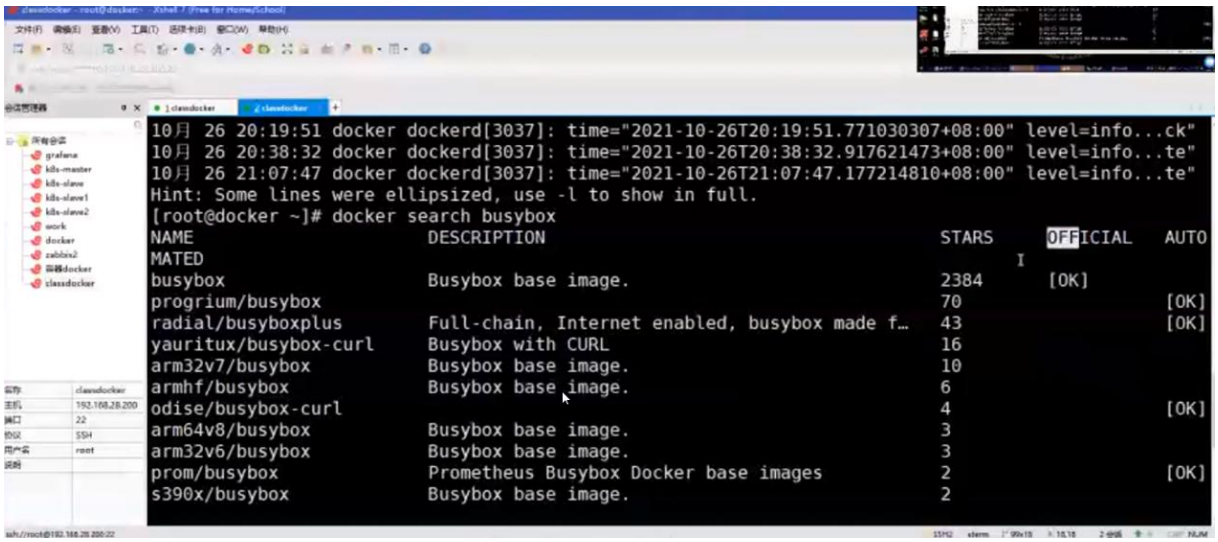
docker cp [文件目录 容器 ID:内部路径]

docker cp [容器 ID:内部路径 文件目录]

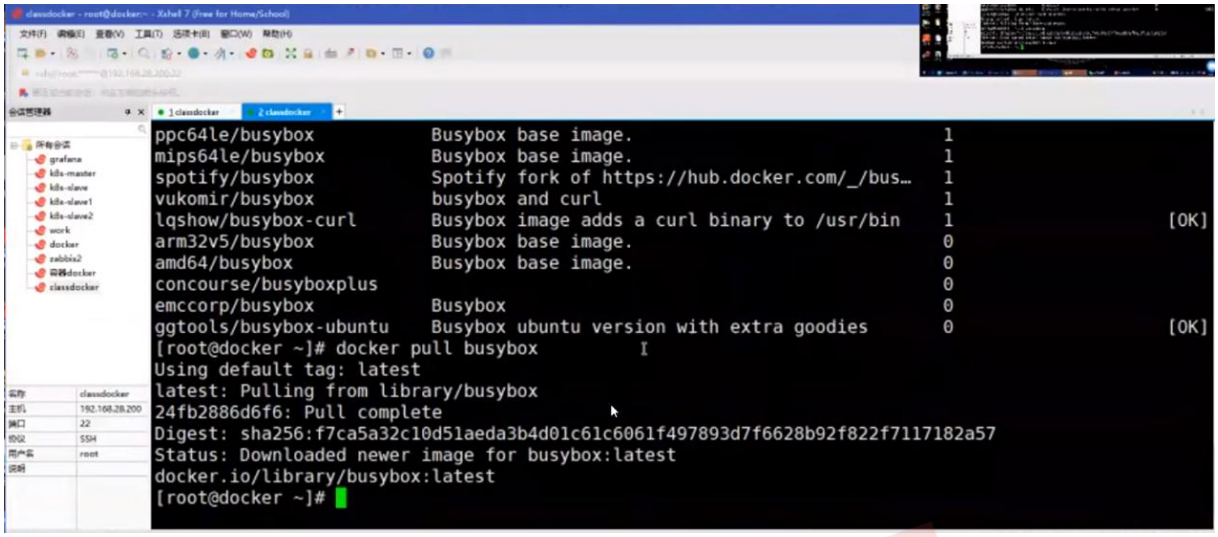
试用水印

二、案例一

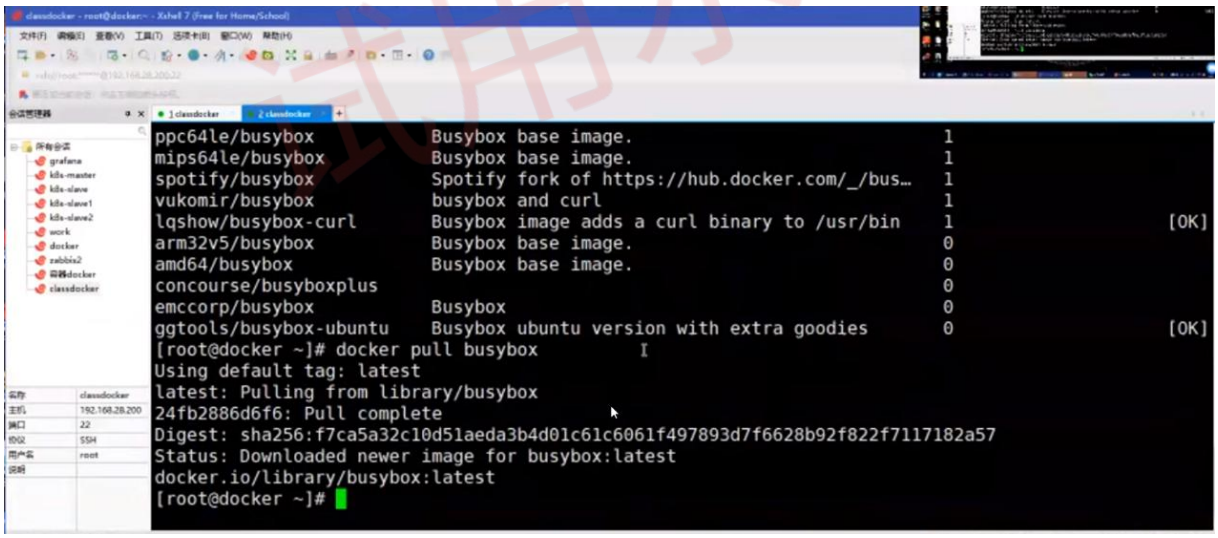
1.搜索 busybox 的镜像，并确保为官方镜像。



2.docker 下载该镜像 busybox 的镜像。

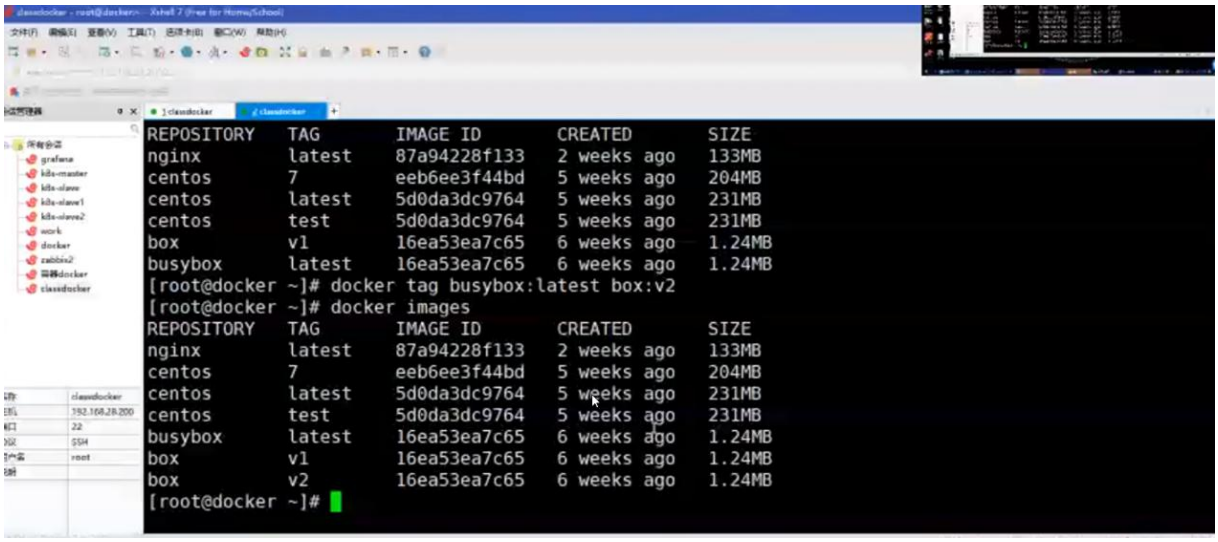


3.给镜像创建一个软链接并改名 box:v1。

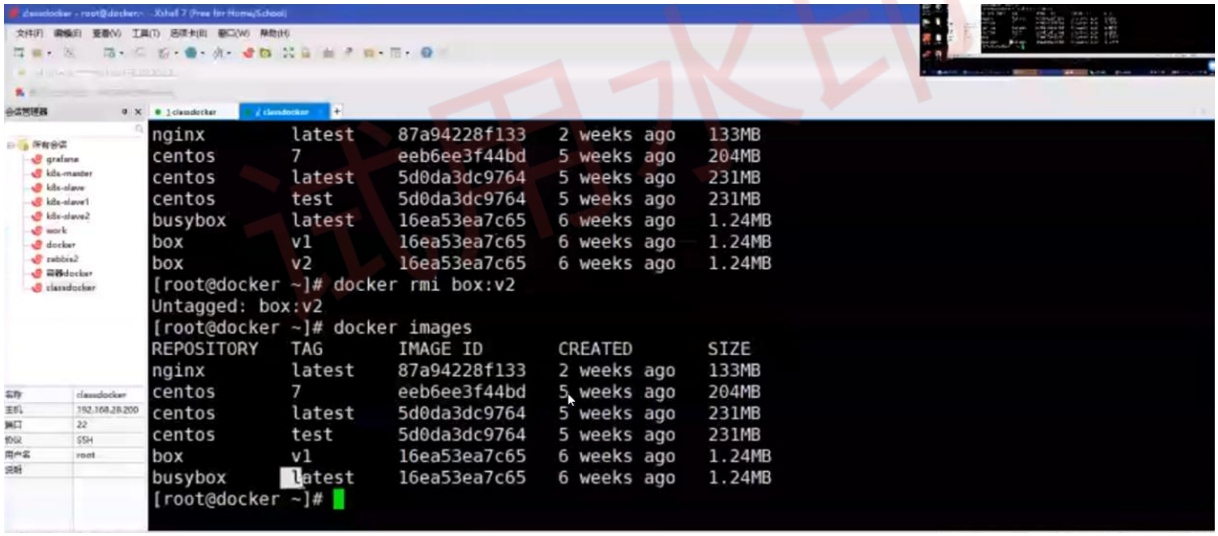


4.再创建一个软链接改名为 box:v2。

试用水印



5.删除 box:v2 镜像。



操作流程

(1)启动虚拟机，检查防火墙和 Slinux 是否关闭

[root@docker ~] systemctl status firewalld

[root@docker ~] getenforce 0

(2)检查 docker 是否启动

[root@docker ~] systemctl status docker

(3)查看 busybox 镜像

[root@docker ~] docker search busybox

(4)下载 busybox 镜像并检查是否成功

[root@docker ~] docker pull busybox

[root@docker ~] docker images

(5)给镜像创建一个软链接并改名 box:v1 并检查是否成功

[root@docker ~] docker images

[root@docker ~] docker tag busybox:latest box:v1

查看 ImageID,如果一样则软连接成功

(6)再创建一个软链接改名为 box:v2。

[root@docker ~] docker tag busybox:latest box:v2

(7)删除 box:v2 镜像

[root@docker ~] docker rmi box:v2

至此，第一个案例就此完成。

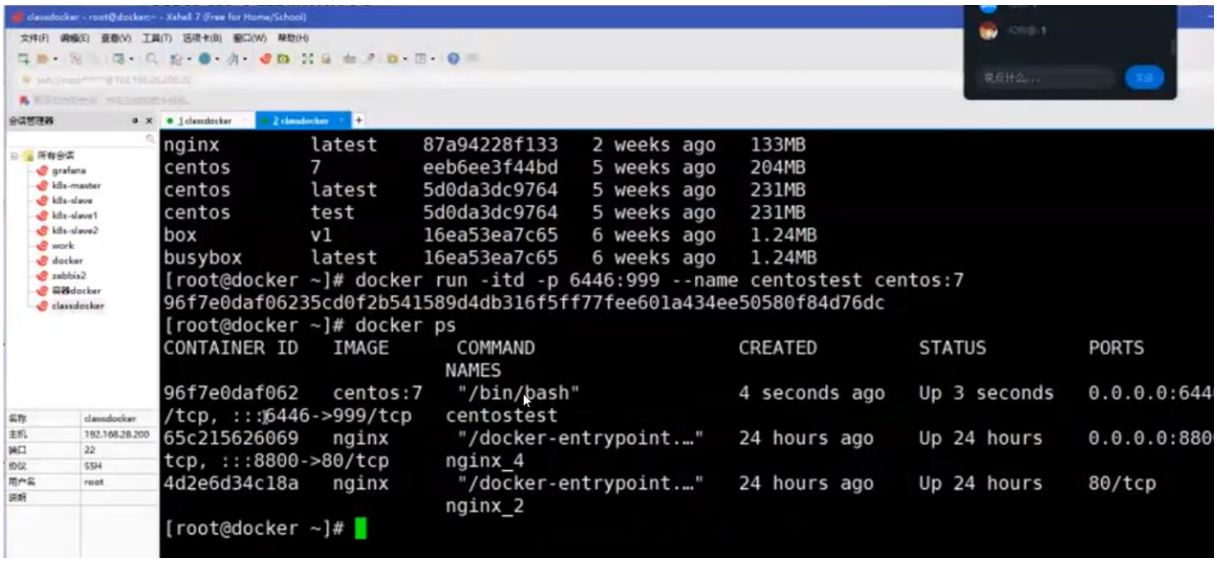
三、案例二

[root@docker ~]

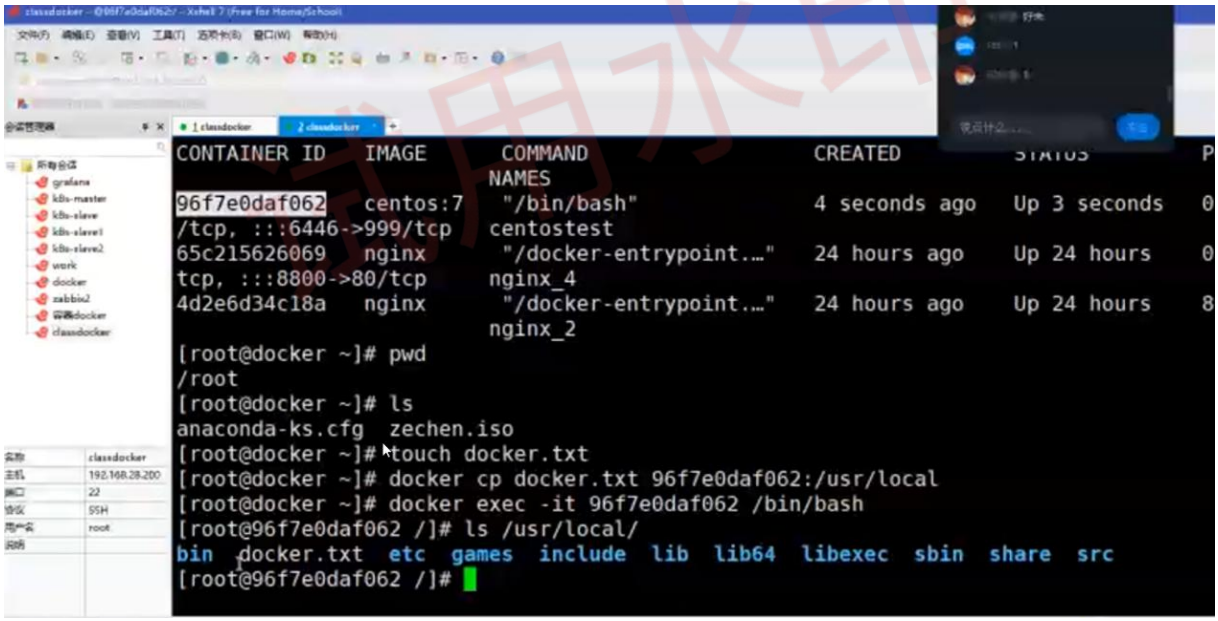
(没有文件自行创建)

1.下载 centos:7 镜像。并用该镜像启动容器，命名为 centostest,暴露 6446 宿主机端口，映射容器内 999 端口

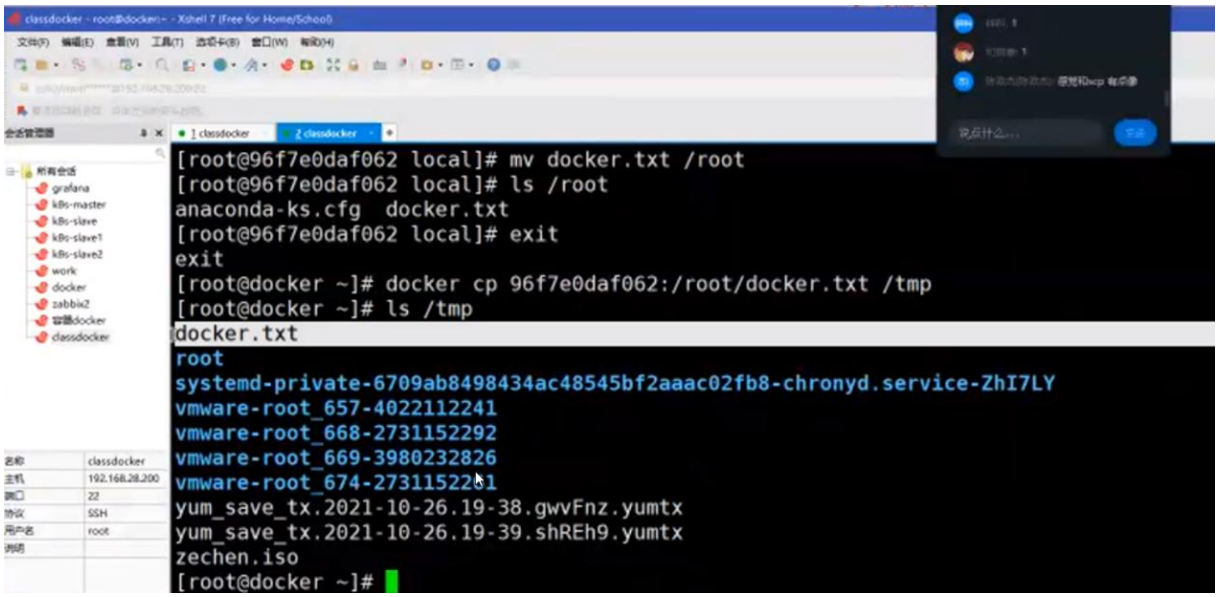
试用水印



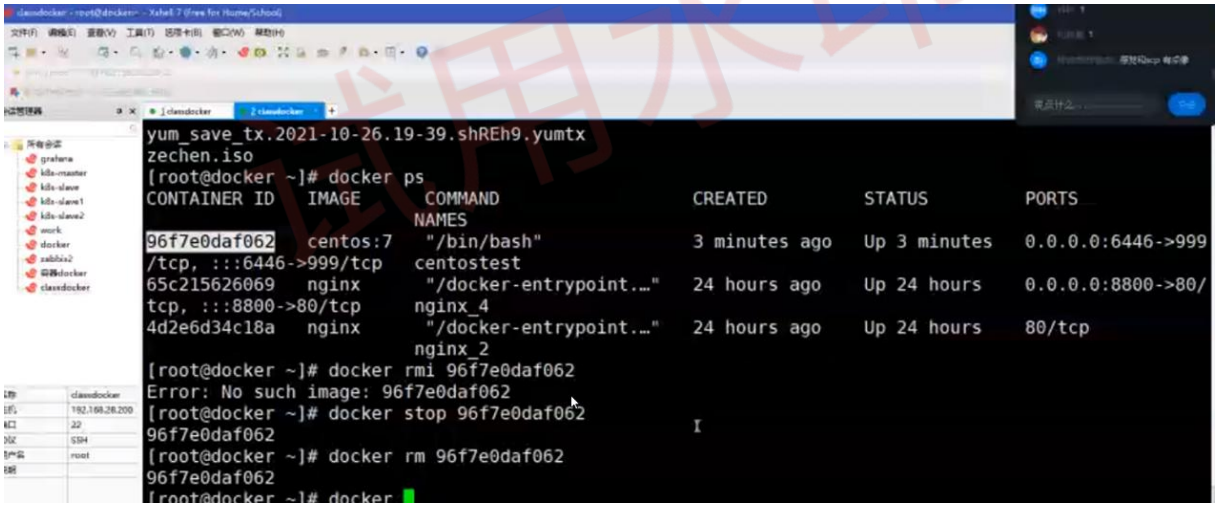
2.将宿主机/root/magegood.txt 复制到容器内/user/local/



3.再从该容器内的/root/magestudy.txt 文件复制到宿主机/tmp



4.删除该容器。



操作流程

(1)用该镜像启动容器，命名为 centostest,暴露 6446 宿主机端口，映射容器内 999 端口

[root@docker ~] docker run -itd -p 6449:999 --name centostest centos:7

参数解释：

-itd:固定写法

-p:暴露的端口号

--name:修改名字

(2)查看容器是否启动成功

[root@docker ~] docker ps

(3)将宿主机/root/magegood.txt 复制到容器内/usr/local/

[root@docker ~] touch docker.txt

[root@docker ~] docker cp docker.txt 96f7e0d:/usr/local

验证

[root@docker ~] docker exec -it 96f7e0d /bin/bash

[root@docker ~] ls /usr/local

(4)再从该容器内的/root/magestudy.txt 文件复制到宿主机/tmp

[root@docker tmp] ls /tmp

[root@docker tmp] docker cp 96f7e0d :/root/docker.txt /tmp

(5)删除该容器并检查

删除前需要停掉

[root@docker ~] docker stop 96f7e0d

[root@docker ~] docker rmi 96f7e0d

[root@docker ~] docker ps

注意：使用 search 搜索 centos:7 是找不到官方版本的

[root@docker ~] docker search centos:7

[root@docker ~] docker search centos tag 即可搜索官方的

四、Dockerfile

试用水印



1.概述

dockerfile 可以理解为一个制作镜像的脚本，但远没有脚本复杂。他根据某种格式自定义内容，就可以快速创建出需求的镜像。

docker 容器启动的时候在最上层挂载了一个可写层，比如说我在容器里面创建一个文件，这个文件是存放在可写层的，这时候容器要是销毁了，那么我们对容器的一些写入操作也就没了，我这个文件也会随着容器销毁而销毁了，我们要是想要保存我们对容器的一些写入操作的话，可以使用 commit 命令然后将容器制作成一个镜像，这样下次 run 起来该镜像的时候，我们之前的写入操作就还存在了。

除了使用 commit 方式制作镜像，还有一种方式就是编写 Dockerfile 然后使用 build 命令来制作镜像了。

2.Dockerfile 的规则

格式：

是注释

指令建议要大写，内容小写

试用水印

执行顺序：

docker 是按照 Dockerfile 指令顺序依次执行的，也就是说从上到下

3.指令

1）FROM：底层镜像（如系统）

这个 FROM 指令是 dockerfile 的第一个指令，然后指定了基础镜像，后面的所有指令都是运行在该基础镜像环境上的 MAINTAINER 该指令是描述的维护者信息。

底层的系统镜像用的是什么，使用 from 指定，绝大部分情况都用的底层有个系统或者基础的环境用的是什么，对 docker 不熟练使用 centos 镜像即可。

2）RUN：构建时容器内运行的命令。

RUN 指令用于在容器中执行命令。我们常用来安装基础软件。

镜像安装软件依赖包都可以放在 run 中。

3）COPY：复制 docker 目录中的文件到镜像中。

COPY 指令类似 ADD 指令，但是 ADD 指令范围更广些，ADD 能够自动解压文件，能够访问网络资源，而 COPY 指令做不到。

非目录需要重新指定，放在目录中非常便利，属于好的一种习惯，值得推荐使用目录。

4）ADD：复制 docker 目录中的文件到镜像中。（过程可以解压）

ADD 指令是用来将宿主机某个文件或目录放到（复制）容器某个目录下面。

官方不推荐 ADD,高级复制功能，需求不精准，推荐使用 COPY。

5）EXPOSE：声明开放端口。

EXPOSE 指令用于暴露容器里的端口，我们在 3.5 里面演示过了，nginx 暴露的端口是 80，但是启动容器的时候需要指定宿主机端口来映射你暴露的端口。需要暴露多个端口的话可以使用多个 EXPOSE，也可以一个 EXPOSE 指令后面跟多个端口，端口之间用空格隔开。

声明不是变更，变更使用-p 构建容器时候使用。

试用水印

6）ENV：设置环境变量。

ENV 指令是用于设置环境变量的 。

底层环境变量需要需提前设置。

7）CMD：容器启动时执行的命令，最多只能执行一条。

CMD 指令是你在容器启动的时候帮你运行的命令，而 RUN 这个指令是构建镜像的时候帮你运行的命令。

容器启动时执行命令，最多执行一条。

8）WORKDIR：声明工作目录。类似 cd。

WORKDIR 是指下面的指令都在 WORKDIR 指定目录下面工作，这个与 linux 里面的 cd 差不多。

切换目录使用 WORKDIR。

做容器轻量级最好，比较小就很好，使用一条命令就不要使用两天命令。

3.Dockerfile 步骤

1、创建 dockerfile 目录，名字可以自定义。在大目录里面创建各种小目录和文档，比较有层次。

2、在该目录中编辑 dockerfile。

3、用 COPY 或 ADD，需要把被 COPY 文件提前放到 dockerfile 目录中。

4、官方推荐用 COPY，如果需要过程中解压，用 ADD。

5、CMD 字段，需要用[“命令”,”选项 1”]这种格式书写。强制格式写法，必须使用[“命令”,”选项 1”]，类似于 python 的数组。

6、CMD 字段只能写一条命令，且这条命令尽量是前台执行的命令。如果是后台命令，这条命令结束，docker 就会自动关闭。

区别：前台执行命令和后台执行命

1）后台命令：

当在前台运行某个作业时，终端被该作业占据；而在后台运行作业时，它不会占据终端。可以使用&命令把作业放到后台执行。当在后台执行命令时，可以继续使用你的终端做其他事情。适

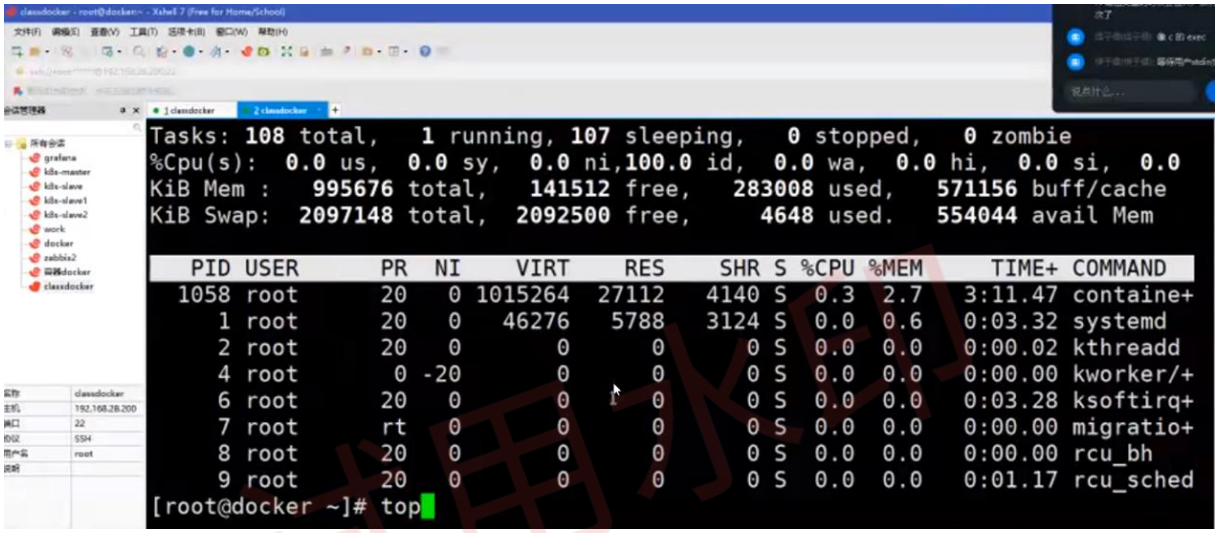
试用水印

合在后台运行的命令有 find、费时的打印作业、费时的排序及一些 shell 脚本。在后台运行作业时要当心：需要用户交互的命令不要放在后台执行。

注意：经常操作的 shell 命令都属于后台命令。

2）前台命令：

top 属于前台命令没有特殊操作会一致出现。



五、dockerfile 操作测试

阿里云 开发者训练营

dockerfile

用dockerfile构建镜像：
docker build -t（设置要构建镜像的名字） .

再运行镜像：
docker run -itd -p 8800:80 [image:tag]

查看镜像：
docker ps -a 确认是启动状态

测试：
宿主机用浏览器访问IP+port 看是否启动成功。

1.用 dockerfile 构建镜像：

docker build -t（设置要构建镜像的名字,最后要加.）

创建基础镜像

Docker 提供了两种方法来创建基础镜像

一种是通过引入 tar 包的形式，

另外一种是通过一个空白的镜像来一步一步构建

scratch 是 Docker 保留镜像，镜像仓库中的任何镜像都不能使用这个名字，使用 FROM scratch 表明我们要构建镜像中的第一个文件层。

如果我们有一个 Linux 下可执行的二进制文件，可以构建一个简单的镜像，仅执行这个二进制。构建的过程很简单，执行如下命令。

$ docker build -t chello:0.1 .

通过 docker images 命令可以本地的镜像。

$ docker images

试用水印

2.再运行镜像：

docker run -itd -p 8800:80 [image:tag]

Docker 中的容器运行在操作系统中，共享了操作系统的内核。对于在 Mac、Windows 平台下，则是基于 Linux 虚拟机的内核。

而 Linux 内核仅提供了进程管理、内存管理、文件系统管理等一些基础的管理模块。除此之外，我们还需要一些 Linux 下的管理工具，包括 ls、cp、mv、tar 以及应用程序运行依赖的一些包。因此我们就需要首先构建一个 Minimal 的操作系统镜像，在此基础上构建 Python 环境，再构建应用镜像。这样就实现了镜像文件分层，今后如果我们需要更新 Python 版本，那么只需要对这一层进行更新就可以。

3.查看镜像：

docker ps -a 确认是启动状态

docker 的镜像实际上由一层一层的文件系统组成，这种层级的文件系统就是上文说到的 UnionFS。在 Docker 镜像的最底层是 bootfs。这一层与我们典型的 Linux/Unix 系统是一样的，包

含 boot 加载器和内核。

当 boot 加载完成之后整个内核就都在内存中了，此时内存的使用权已由 bootfs 转交给内核，此时系统也会卸载 bootfs。

Docker 在 bootfs 之上的一层是 rootfs（根文件系统）。rootfs 就是各种不同的操作系统发行版，比如 Ubuntu，Centos 等等。Docker 核心技术与实现原理 这篇文章，作者阅读了 rootfs 的规范，指出构建 rootfs 一些必须的文件夹。

4.测试：

宿主机用浏览器访问 IP+port 看是否启动成功。

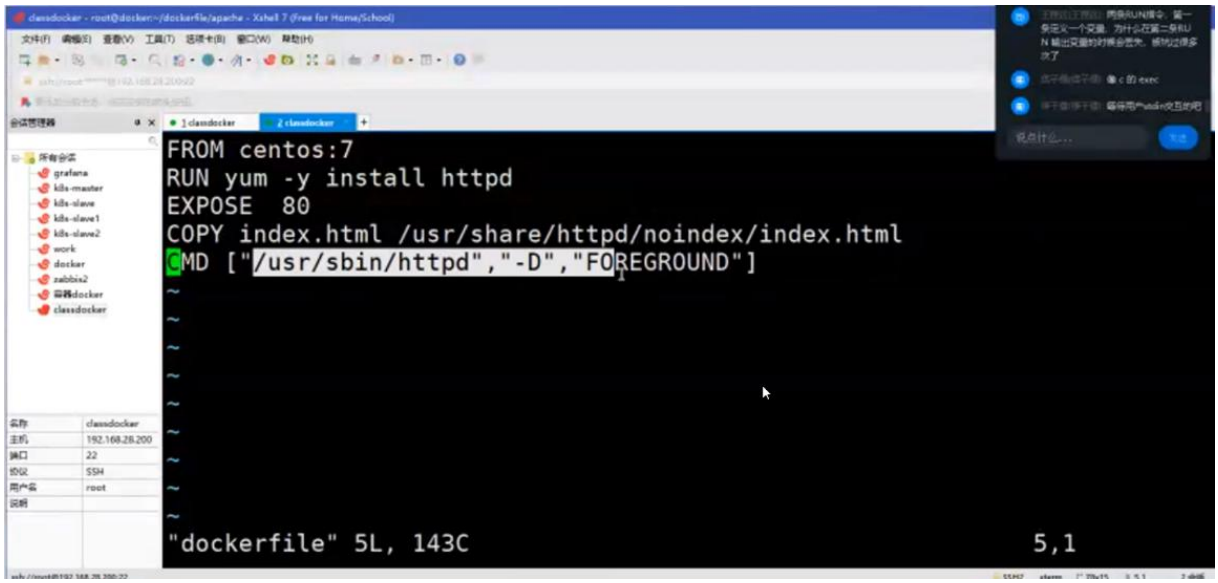
六、dockerfile 案例 1

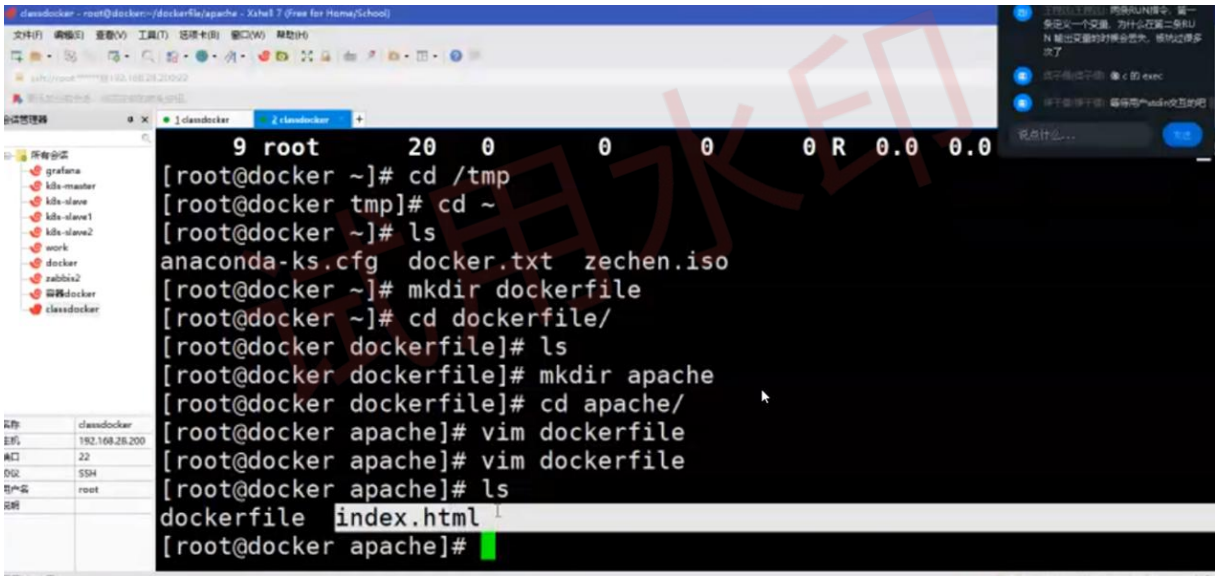
试用水印



1）用 dockerfile 创建并启动一个 centos 的 apache 镜像。指定自定义内容。

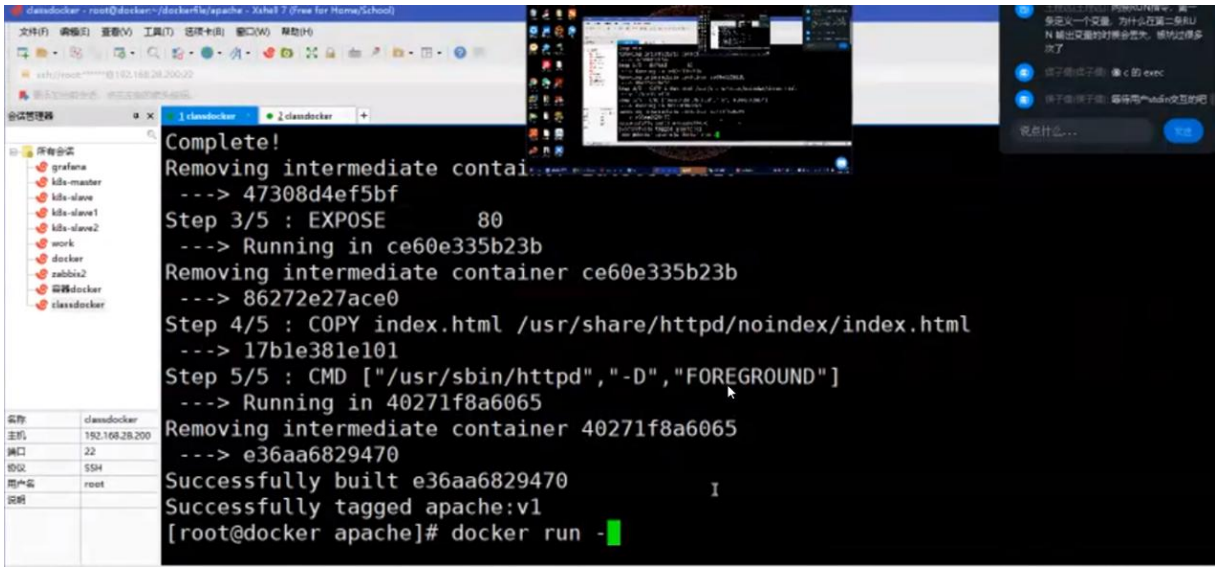
```
FROM centos:7
RUN yum -y install httpd
EXPOSE 80
COPY index.html /usr/share/httpd/noindex/index.html
CMD ["/usr/sbin/httpd","-D","FOREGROUND"]
```





2）构建镜像

[root@docker ~] docker build -t apache :v1 .



看到 successfully 即安装成功

3）验证

[root@docker ~] docker run -itd -p 345:80 apache:v1

[root@docker ~] docker ps

通过访问宿主机 ip 端口，查看 true，防火墙关闭



七、dockerfile 案例 2

1）用 dockerfile 创建并启动一个 centos7 的 nginx 镜像。

试用水印

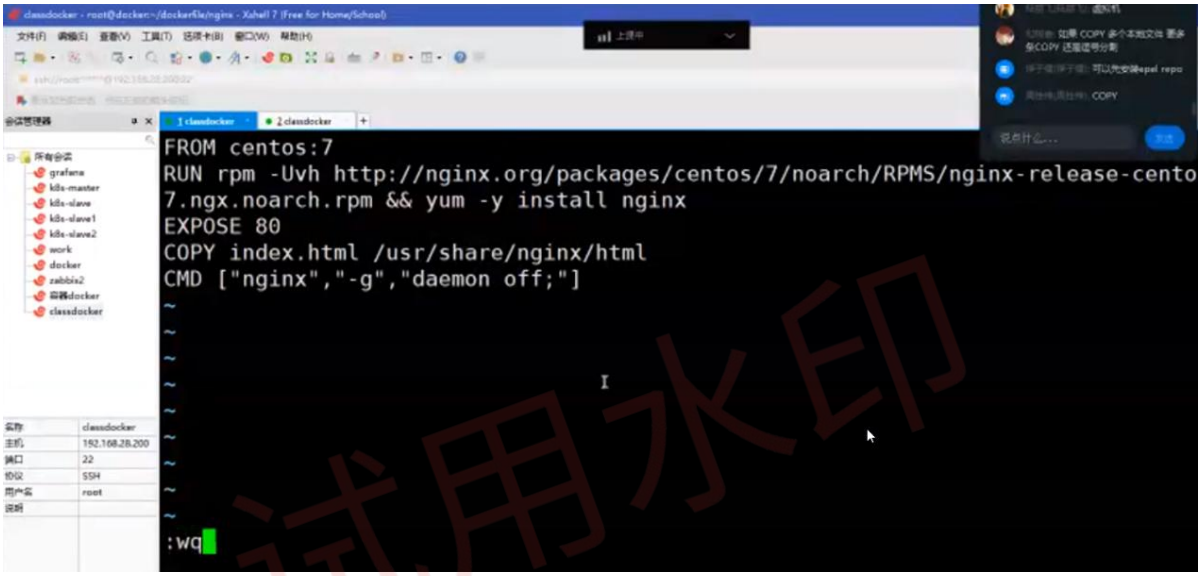
FROM centos:7

RUN?rpm?-Uvh?http://nginx.org/packages/centos/7/noarch/RPMS/nginx-release-centos-7-0.el7.ngx.noarch.rpm

RUN yum -y install nginx

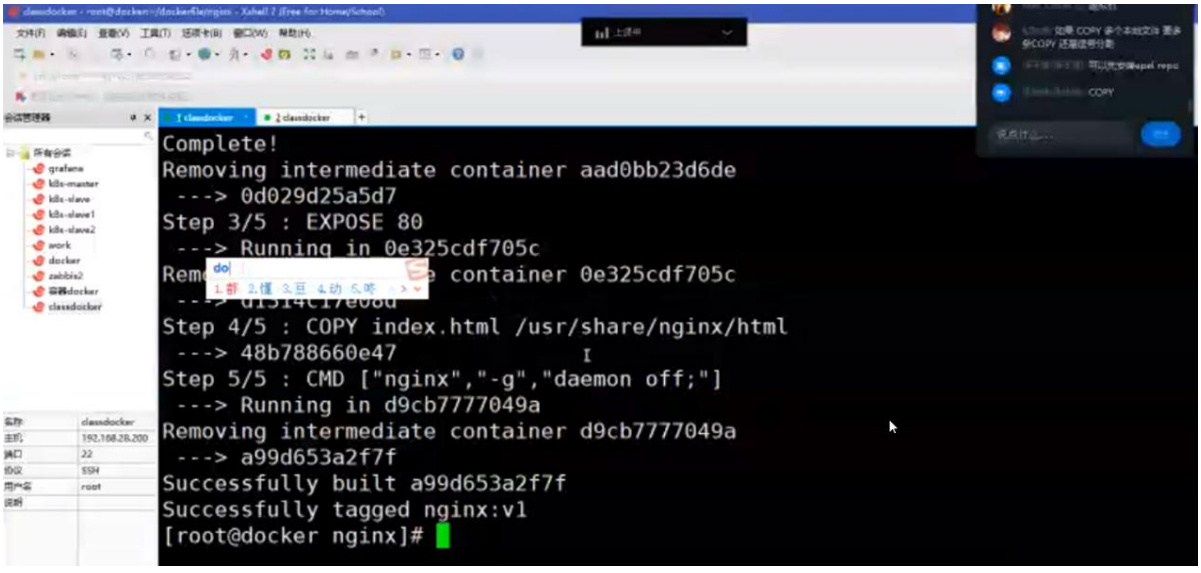
EXPOSE 80

CMD ["nginx","-g","daemon off;"]



2）创建

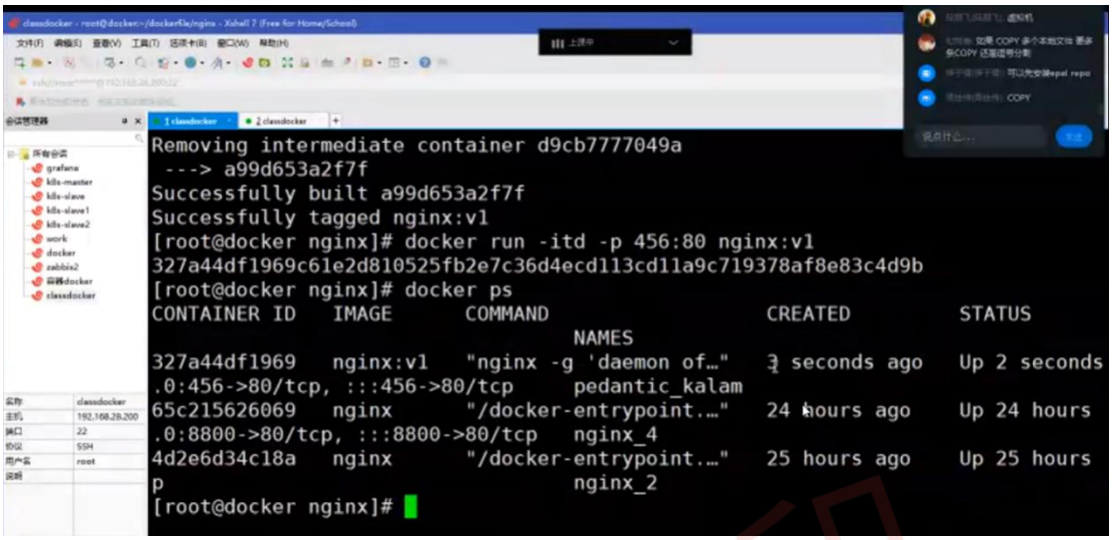
[root@docker nginx]docker bulid -t nginx:v1 .



试用水印

3）创建容器

[root@docker dockerfile] docker run -itd -p 456:80



4)验证

通过访问宿主机的 ip+端口，查看 Nginx，防火墙关闭。



试用水印

docker 实际工作案例实现

演讲人：马哥教育李老师 阿里云 IoT 技术专家

视频链接：https://developer.aliyun.com/learning/course/892



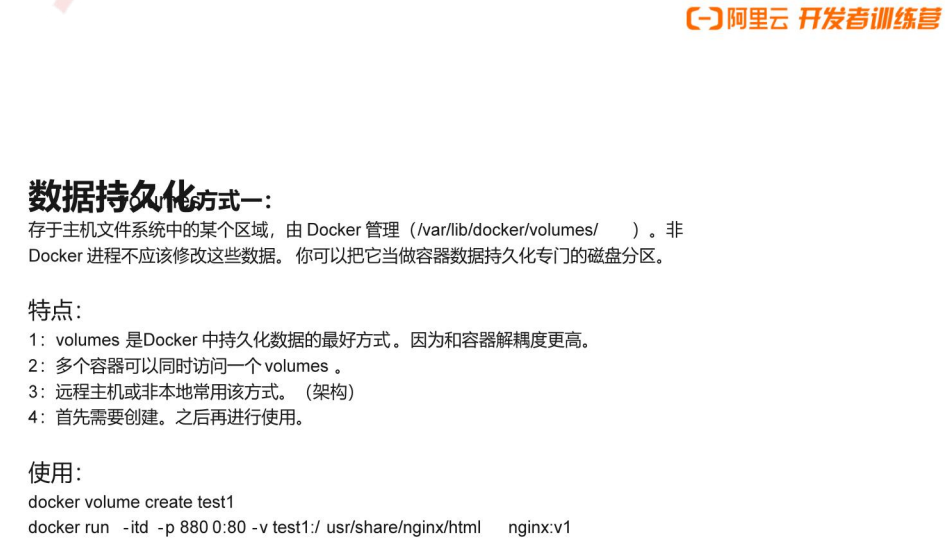


一、数据持久化

我们什么情况下要做数据持久化呢？一定是在做容器之前先预判好哪些文件是要永久存储的，而不会跟着它容器的一个生命周期而消失。



比如说配置文件、日志文件、缓存文件或者应用数据等等。数据初始化有三种类型。第一种volumes，这个是最推荐的，也是最好的一种方式。第二种是 bind—mount，第三种是 tmpfs。



试用水印

方式一：volumes

是官方比较推荐也是大型的集群比较常见的一种方式。可以理解为在自己的宿主机或者云端或者在某一个区域创建一块磁盘专门去存放容器里的数据或文件。把这个容器里边的数据或者文件还有目录等都规划好，再去启动容器。正常在老一些的版本里边首先必须要去创建 volumes，否则是没有办法创建成功的。新版本好像不写命令也可以创建成功，具体可以查询官方关于 volumes 的文档。

使用：

docker volume create test1

docker run -itd -p 8800:80 -v test1:/usr/share/nginx/html nginx:v1

创建：docker volume create

删除某个卷：docker volume rm 卷名

删除所有未使用的卷：docker volume prune

列出所有卷：docker volume ls

查看某个卷的信息：docker volume inspect 卷名

挂载到容器：-v 或–volume。如果是 Docker17.06 或更高：推荐使用–mount。（同 bind mount）

挂载类型：key 为 type，value 为 bind、volume 或 tmpfs

挂载源：key 为 source 或 src，对于命名卷，value 为卷名，对于匿名卷，则忽略

容器中的挂载点：key 为 destination、dst 或 target，value 为容器中的路径

读写类型：value 为 readonly，没有 key

volume-opt 选项，可以出现多次。比如 volume-driver=local,volume-opt=type=nfs,…

第一个域：对于命名卷，为卷名；匿名卷，则忽略，此时会创建匿名卷

第二个域：容器中的挂载点

第三个域：可选参数，由’,'隔开，如 ro

-v 或—volume：由 3 个域组成，’:’分隔

—mount：由多个’,’隔开的键值对组成

试用水印

数据持久化方式二：

将宿主机中的文件、目录mount 到容器上。

特点：

1：它更像1对1的夫妻。耦合度较高。

2：一些监控类container ，通过读取宿主机固定文件中的数据实现监控等

3：常用于临时共享文件（如配置文件等）或源码文件。

使用：

docker run -itd -p 8801:80 -v /var/log/cont/apache1:/var/log/httpd/ apache:new2

方式二：bind—mount

将宿主机中的文件、目录 mount 到容器上。质上是宿主机、container 之间共享宿主机文件系统。这种持久化方法更导致 container 与宿主机的耦合过于紧密，所以不推荐使用。

使用：

docker run -itd -p 8801:80 -v /var/log/cont/apache1:/var/log/httpd/ apache:new2

挂载到容器：-v 或—volume。如果是 Docker17.06 或更高：推荐使用—mount。（同 volumes）

-v 或–volume：由 3 个域组成，’ :'分隔

第一个域：对于命名卷，为卷名；匿名卷，则忽略，此时会创建匿名卷

第二个域：容器中的挂载点

第三个域：可选参数，由’ ,'隔开，如 ro

–mount：由多个’ ,'隔开的键值对=组成：

挂载类型：key 为 type，value 为 bind、volume 或 tmpfs

挂载源：key 为 source 或 src，value 为主机中文件或目录的路径

容器中的挂载点：key 为 destination、dst 或 target，value 为容器中的路径

试用水印

- 读写类型：value 为 readonly，没有 key
- bind-propagation 选项：key 为 bind-propagation，value 为 rprivate、private、rshared、shared、rslave 或 slave
- 一致性选项：value 为 consistent、delegated、cached。这个选项仅仅适用于 Docker for Mac
- –mount 不支持 z 和 Z（这个不同于-v 和—volume）

阿里云 开发者训练营

数据持久化方式一：

将数据存于宿主机内存中。

特点：

1：仅限Linux系统。

2：较少用，常见用于对于访问有大量读写，或安全层面考虑。

3：不会持久化，生命周期限于容器启动时。

4：多容器无法共享同一个tmpfs

使用：

docker run -itd -- name tmptest -- tmpfs / root nginx:latest

试用水印

方式三：tmpfs

将数据存于宿主机内存中。docker 可将用户名与密码等敏感数据保存在某个数据库中，当启动需要访问这些敏感数据的 container 或者 service 时，docker 会在宿主机上创建一个 tmpfs，然后将敏感数据从数据库读出写到 tmpfs 中，再将 tmpfs mount 到 container 中，安样能保证数据安全。当容器停止运行时，则相应的 tmpfs 也从系统中删除。

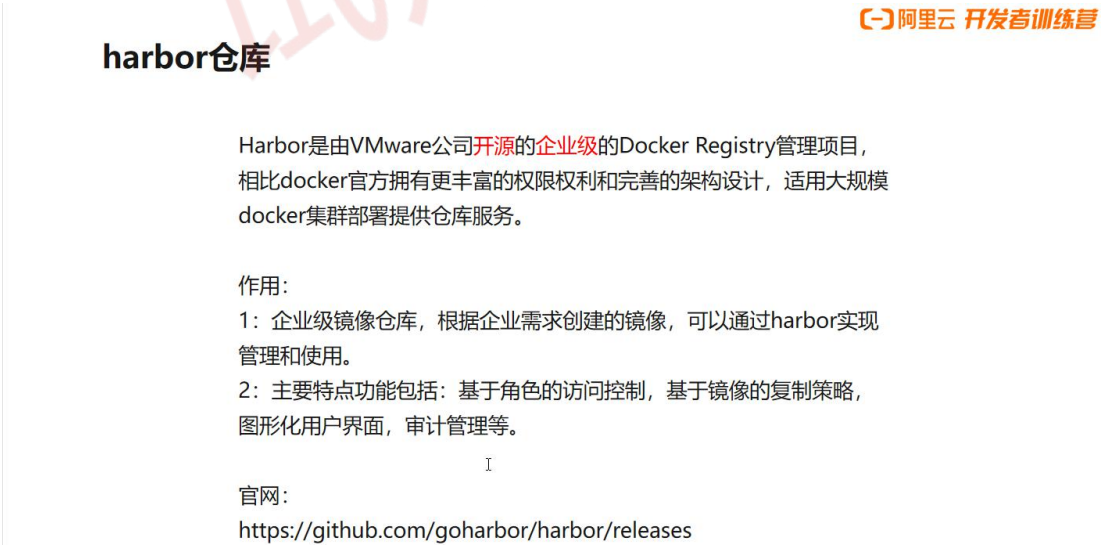
docker run -itd --name tmptest --tmpfs /root nginx:latest

二、harbor 仓库



镜像获取有两种方式：

- 第一种方式是从官方的镜像仓库获取，search 命令从官方镜像查找已有的镜像，
- 第二种方式是企业级镜像仓库，公司最常用的一种 harbor。



harbor 的作用 1：企业级镜像仓库，根据企业需求创建的镜像，可以通过 harbor 实现管理和使用。

harbor 的作用 2：主要特点功能包括：基于角色的访问控制，基于镜像的复制策略，图形化用户界面，审计管理等。

试用水印

安装

安装并启动 docker

安装 docker-compose1.18 以上版本。

（docker 单机容器编排工具）

•wget

https://github.com/docker/compose/releases/download/1.29.2/

docker-compose-Linux-x86_64

#从官网下载 docker-compose 二进制文件

•chmod +x docker-compose-Linux-x86_64

#添加执行权限

•mv docker-compose-Linux-x86_64 /usr/bin

#移动到系统指令目录下

试用水印



tar -xvf harbor-offline-installer-v2.3.1.tgz

#解压

•cp harbor/harbor.yml.tmpl harbor/harbor.yml

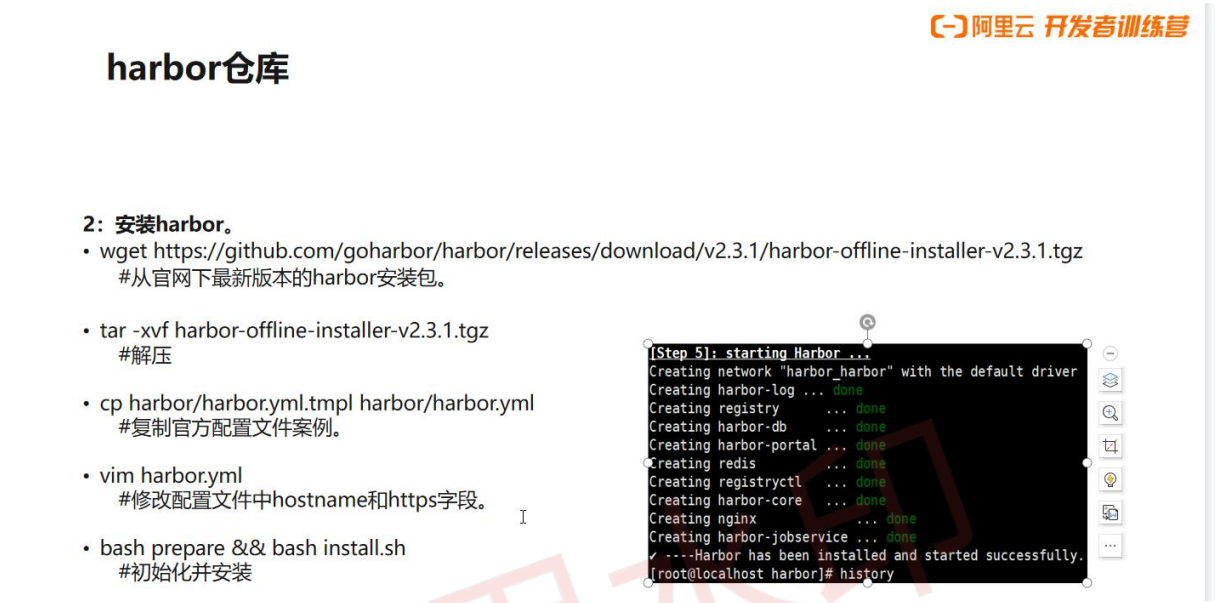
#复制官方配置文件案例。

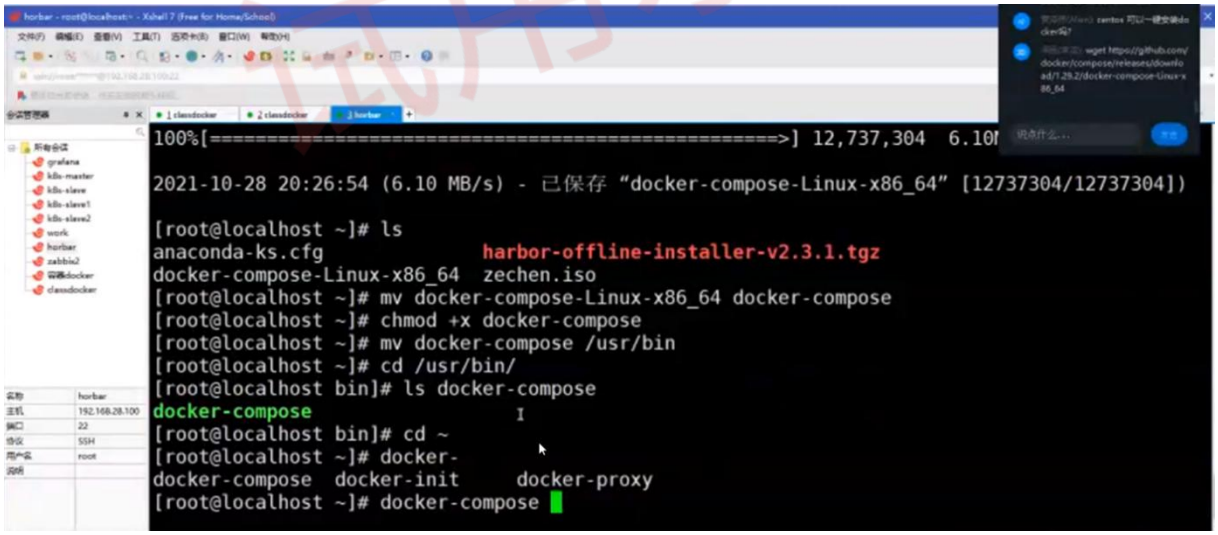
•vim harbor.yml

#修改配置文件中 hostname 和 https 字段。

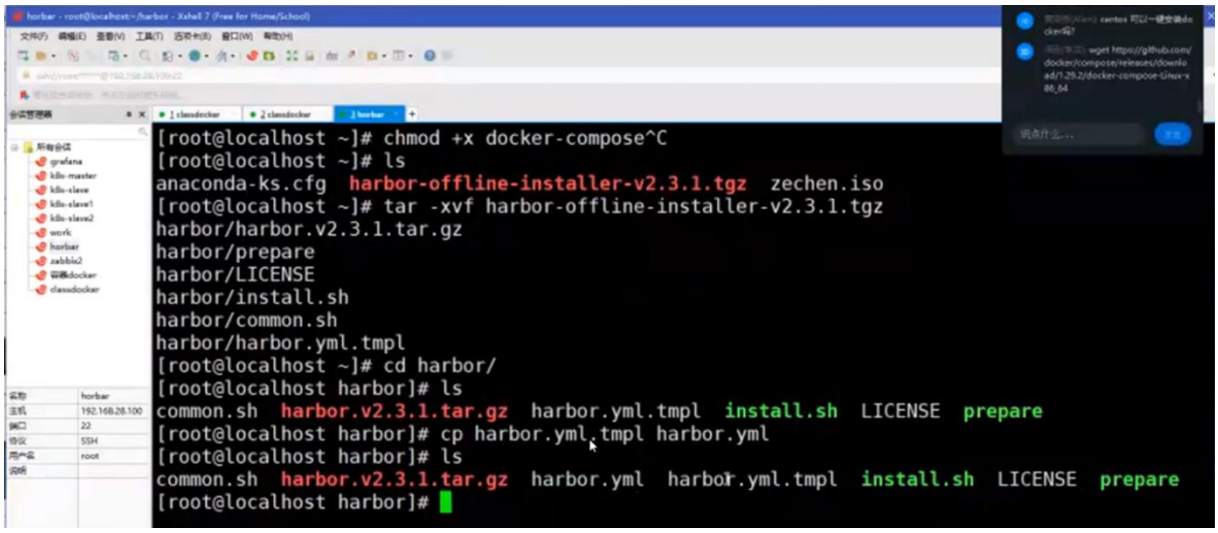
•bash prepare && bash install.sh

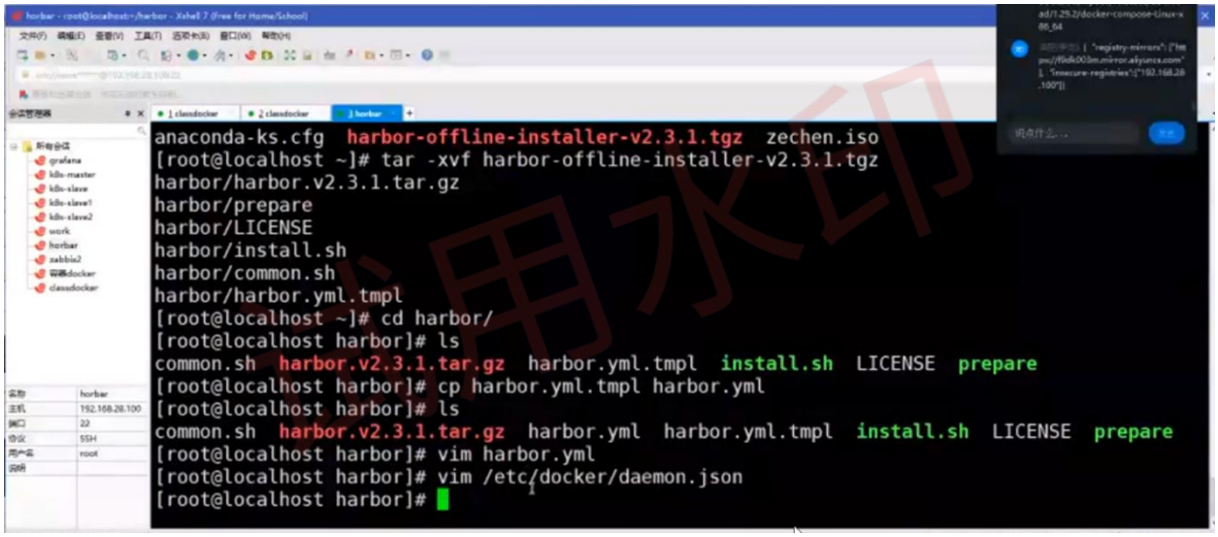
#初始化并安装





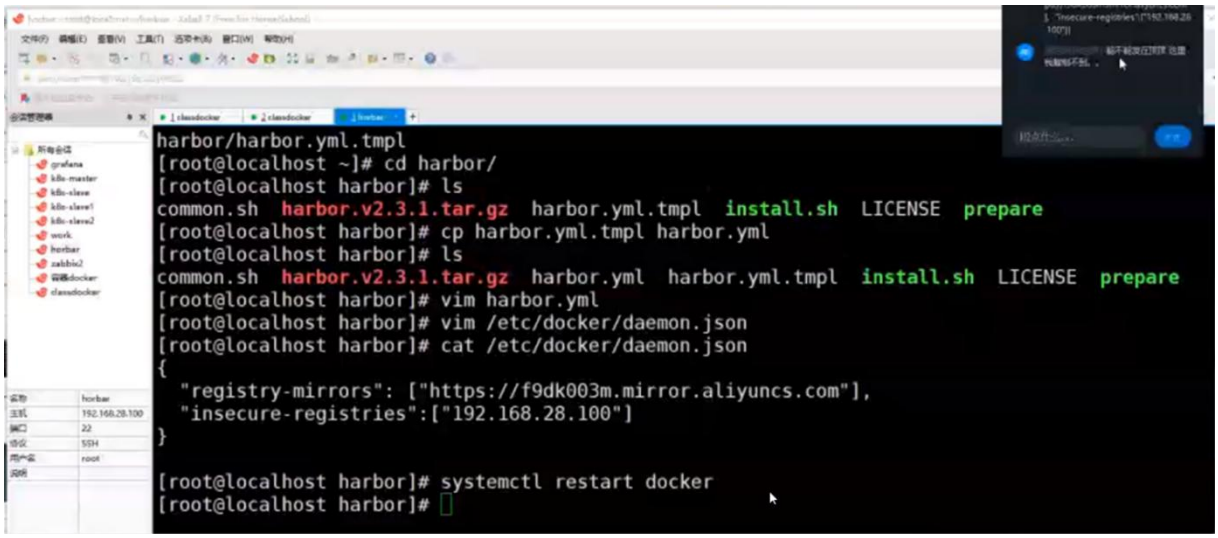
compose 是 docker 的单机容器编排工具。





https 注释，帮助我们实现安全的目的。

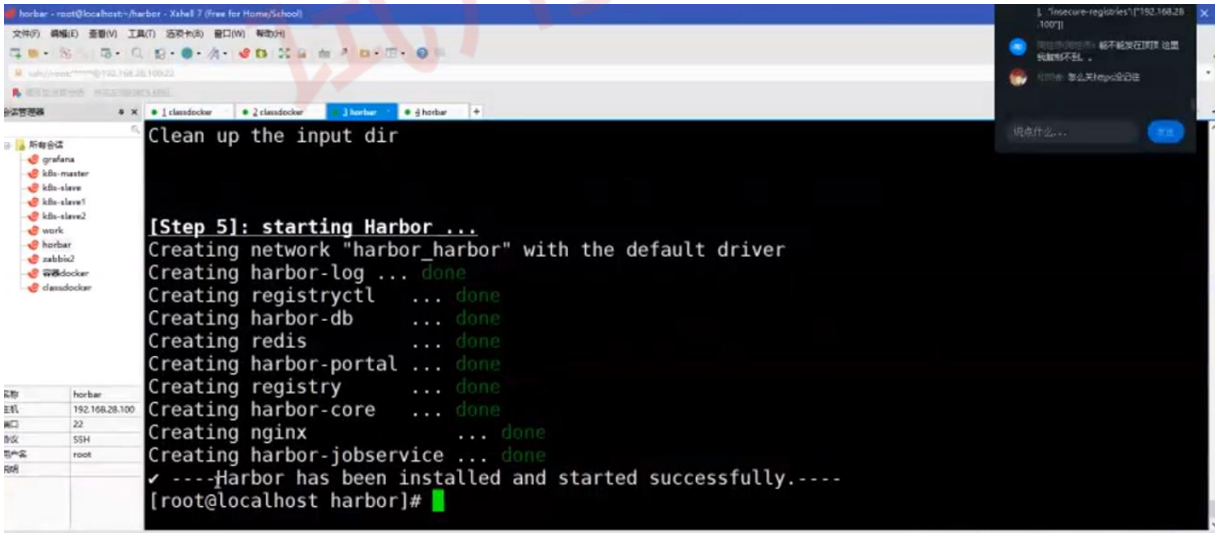
重启 docker。



执行。

保存退出。

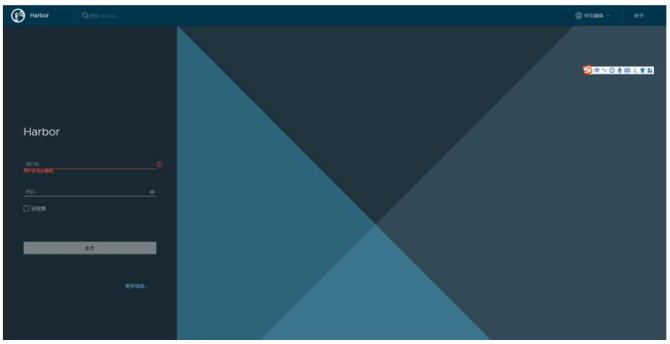
出现 harbor has been stored and start successfully，安装成功。



试用水印

阿里云 开发者训练营

harbor仓库



用户名：admin

密码：Harbor12345

阿里云 开发者训练营

harbor仓库

一：web页面中操作新建项目、新建人员并设置权限、匹配人员和项目。

二：将本地刚刚做的镜像推送到harbor共用项目仓库中。

使用harbor

- docker tag apache:new1 192.168.28.131/library/apache:new1
- vim /etc/docker/daemon.json （信任harbor端IP）

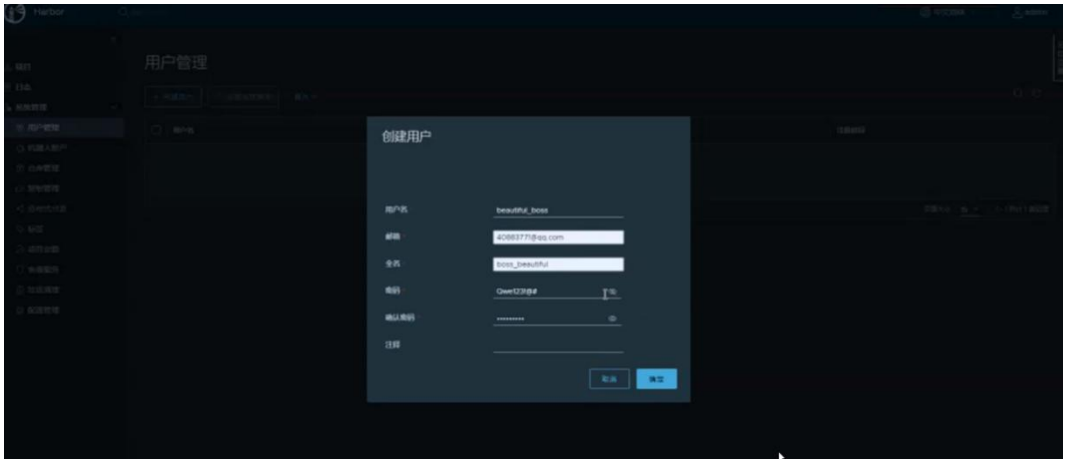
```
{
  "registry-mirrors": ["https://f9dk003m.mirror.aliyuncs.com"],
  "insecure-registries":["192.168.28.131"]
}
```

- docker login 192.168.28.131
- docker push 192.168.28.131/library/apache:new1

三：如何从harbor上边拉取镜像到服务器?

试用水印

应用



名称要和建立名字一样。

实现已经做好镜像放到 harbor 仓库。

将本地刚刚做的镜像推送到 harbor 共用项目仓库中。

dockertagapache:new1 192.168.28.131/library/apache:new1

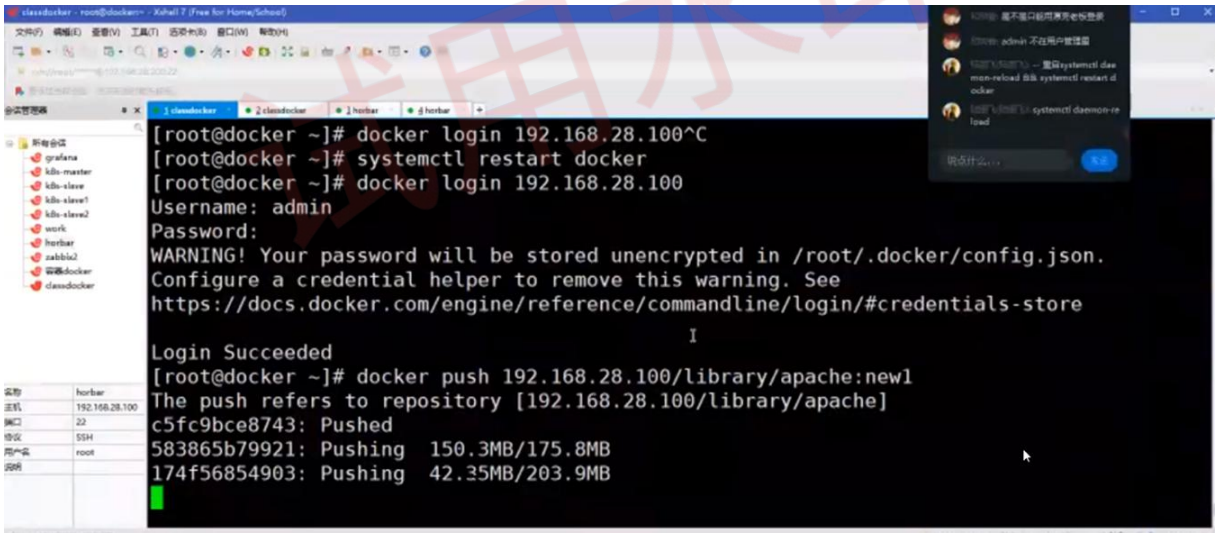
使用 harbor•vim /etc/docker/daemon.json

（信任 harbor 端 IP）

```
{
   "registry-mirrors": ["https://f9dk003m.mirror.aliyuncs.com"],
   "insecure-registries":["192.168.28.131"]
}
```

docker login 192.168.28.131

docker push 192.168.28.131/library/apache:new1



三、微服务



微服务是一种架构的模式，是单体的应用。单体应用拆分成各个模块，充分的去解耦，独立构建部署，互相之间可以协调配合。

微服务的特点，颗粒度小，且专注一件事情。单独的进程。轻量级的通信机制。松耦合，独立部署。



微服务具备四个特性，复杂性，隐匿性，异变性，配合性.

试用水印

1.复杂性

阿里云 开发者训练营

复杂性

困境：系统会随着业务的发展，增加越来越多的特性和功能，使得系统复杂到没有一个人能全面理解，没有一个人敢去修改原有的功能或代码。

微服务：微服务提出以业务边界作为模块划分原则，每个模块独立进程。一个业务由很多独立的小业务组合而成，系统也是由独立的小系统组合而成，这样的好处是每个小系统都很容易被理解，一个大系统可以根据业务组合微服务，或者逐渐发展独立的微服务

代价：微服务为了降低单个微服务的复杂性，导致整体系统的复杂性急剧增加。微服务之间的连接更加复杂，网络通讯不可靠和性能损耗，协议匹配，接口对接和转换，版本协作，微服务注册和发现，编排和调度，分布式业务和数据一致性等等复杂性都是单体架构不需要考虑的。

经验要求、k8s

2.隐匿性

阿里云 开发者训练营

隐匿性

困境：软件在没有应用到业务之前，各种信息和思考大多在每个人的脑海里，很难完全呈现和想象出来。客户只是知道自己想要更多新英雄，但并不知道要具体什么样的英雄；开发设计人员知道怎么做英雄，但又不能完全理解用户需求；运维知道部署服务和上线，但又不能完全理解业务逻辑。单体架构中的各个模块隐藏在大系统的页面下，在交付之前对外界是不可见的。 导致没有人能看清全貌，各自都想把事情做到最好，但组合起来却不是客户想要的东西。

微服务：微服务架构并没有改变软件开发过程中的隐匿性，而是通过缩短从需求到交付这段软件开发周期，减少隐匿时间，来降低软件工程总体的隐匿性。

代价：组织必须要具备自动部署持续交付能力。假如一个系统上线需要3个小时进行部署，如果我们要持续部署，每天都部署一次，那就需要每天拿出3小时做部署，这个成本是不能接受的。

devops

试用水印

3.耦合性

阿里云 开发者训练营

耦合性

困境：假如系统从零开始做的话，头三个月开发会比较慢，因为需要搭建和熟悉一些开发、测试、部署基础设施，随着基础设施和公共组件的完善，接下来的半年到一年开发会加速，但是再往后开发速度又会逐渐降低。因为那些一开始提高开发效率的接口、共享表、依赖组件都变成了复杂网络缠绕在一起，变成了所谓的牵一发而动全身，改一行代码都不知道会影响到什么地方。

微服务：微服务系统的耦合性问题总是在一个可控的范围内。比如微服务独立数据库表结构，那我们根据业务需要改表结构的时候就不需要去考虑会不会影响到其他业务，因为其他业务和这个表结构完全没关系。

代价：单纯从接口协同一致上来说，微服务架构比较糟糕。 单体架构的接口之间配合是相同的编程语言，基本上在编译时就能发现错误。而微服务的接口往往是远程服务，验证增加难度。
运维观战

4.易变性

阿里云 开发者训练营

易变性

困境：比如某个模块的流量突然增加，或者需要大内存，单体架构只能为极少的模块增加整个系统的计算资源，又因为增加整个系统的计算资源成本很高、实施时间长，导致性能需求迟迟不能得到满足。

微服务：微服务要求独立进程，可以完全根据需求定制不同类型的计算资源，更精细化分类的利用内存、IO、CPU。因为小，可以更快水平扩展响应性能需求变化。更关键是，微服务小，强调独立业务价值。小团队直接面对客户需求做决策，所有信息和想法在小范围内快速交流，业务价值流动更容易可见，更快速的响应变化。后期可以弹性扩容。

代价：微服务架构需要改变组织结构小团队充分授权、业务交付模式。对于传统组织而言，这点是最难的，尤其是大公司往往采用层级组织结构。
公司层面

试用水印

总结

阿里云 开发者训练营

总结

1：单体应用的困境，是现在普遍的困境。新的团队有了理念后，会在万丈高楼平地起时就采用该架构，避免单体应用困境。

2：微服务是趋势，现在某些大型公司难以实现只是时间问题。除非它不需要拓展业务和更变业务。

3：18年前后k8s的出现，短短三年就被世界各互联网公司所接受和使用，就足以说明微服务的重要性。对运维人员和开发人员来说，这是个大的挑战和机遇。

4：遇到风口，不进则退。

应用

试用水印

阿里云 开发者训练营

实现

我们最后做一个案例，nginx和php-fpm实现php网页正常运行。
这个案例学通，之后就可以根据该案例知识，做更复杂的架构。
该案例也是运维微服务需要基础掌握的核心内容。

有几个问题需要解决：
1：一个容器只能有一个启动命令（CMD），但这里启动两个服务。
2：多个容器之间如何通信?
3：php-fpm只允许本机通信，多个容器通信需要怎么做?
4：nginx和php-fpm需要访问同一个网页文件，怎么解决?

核心内容



1.Php-fpm 创建镜像准备

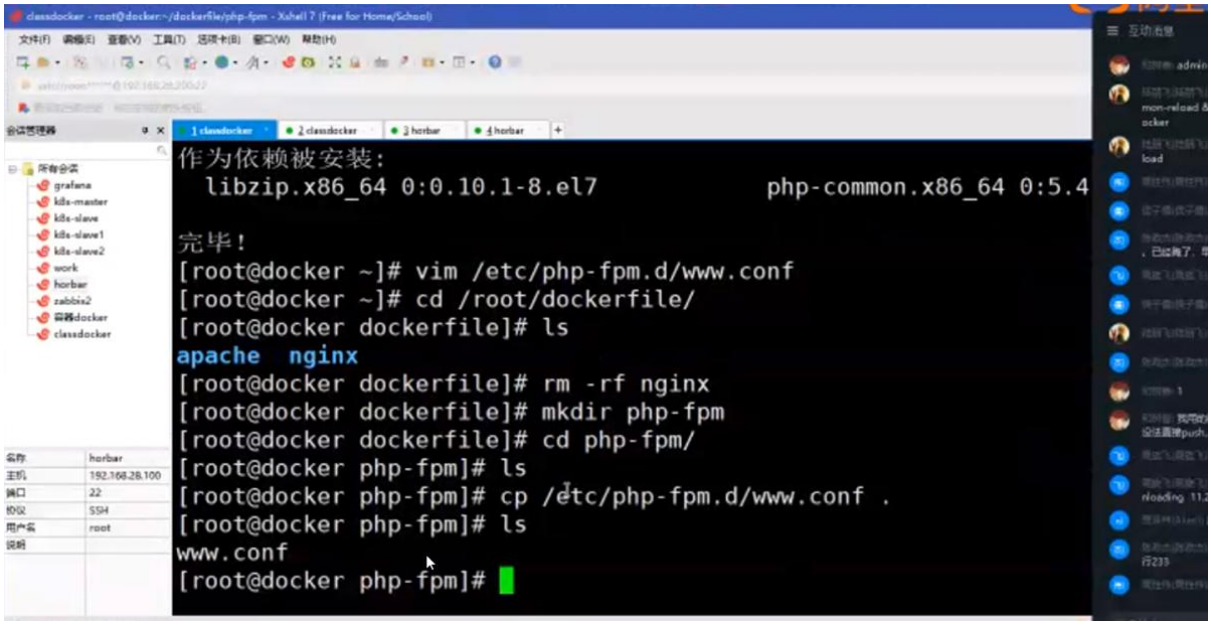
vim/etc/php-fpm.d/www.conf

listen = 0.0.0.0:9000 监听 IP 和端口。

;listen.allowed_clients = 127.0.0.1

mkdir /root/php-fpm

cp /etc/php-fpm.d/www.conf /root/php-fpm/



试用水印

2.php-fpm 的 dockerfile

vim /root/php-fpm/dockerfile

FROM centos:7

RUN yum -y install php-fpm

EXPOSE 9000

COPY www.conf /etc/php-fpm.d/www.conf

CMD ["/usr/sbin/php-fpm","--nodaemonize"]



试用水印

阿里云 开发者训练营

nginx镜像

创建镜像准备：

vim /etc/nginx/conf.d/default.conf

改成php配置

mkdir /root/nginx

cp /etc/nginx/conf.d/default.conf /root/nginx

mkdir /root/nginxweb

vim /root/nginxweb/index.php

<?php echo '<p>true</p>'; ?>

写nginx的dockerfile

vim /root/nginx/dockerfile

FROM centos:7

RUN rpm -Uvh

http://nginx.org/packages/centos/7/noarch/RPMS/

nginx-release-centos-7-0.el7.ngx.noarch.rpm

RUN yum -y install nginx

EXPOSE 80

CMD ["nginx","-g","daemon off;"]

3.nginx 创建镜像准备

vim /etc/nginx/conf.d/default.conf

改成 php 配置

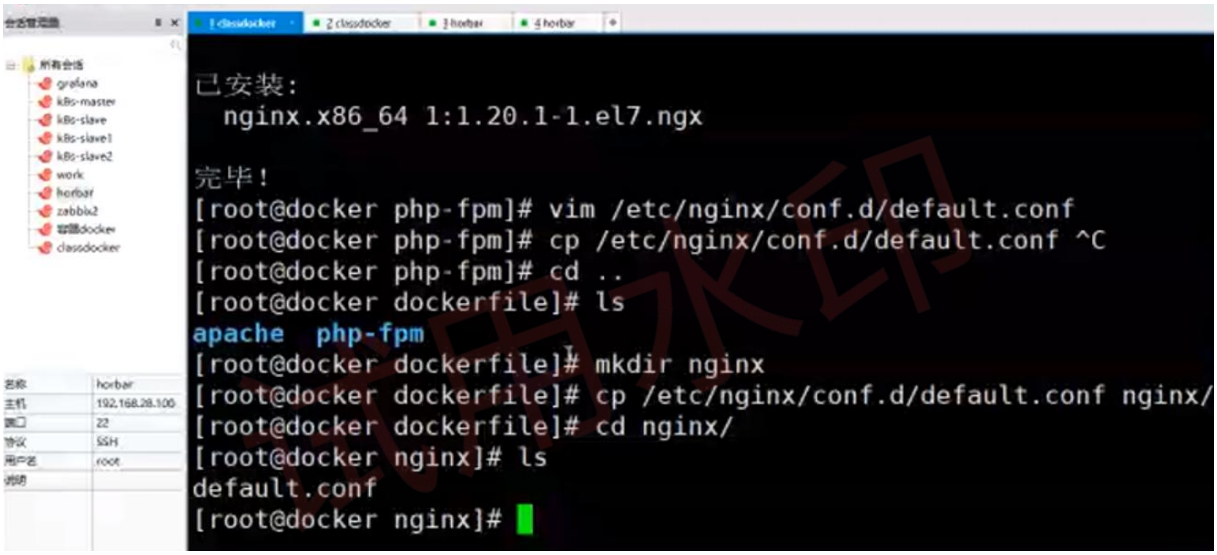
mkdir /root/nginx

cp /etc/nginx/conf.d/default.conf /root/nginx

mkdir /root/nginxweb

vim /root/nginxweb/index.php

<?php echo '<p>true</p>'; ?>



4.写 nginx 的 dockerfile

vim /root/nginx/dockerfile

FROM centos:7

RUN rpm -Uvh

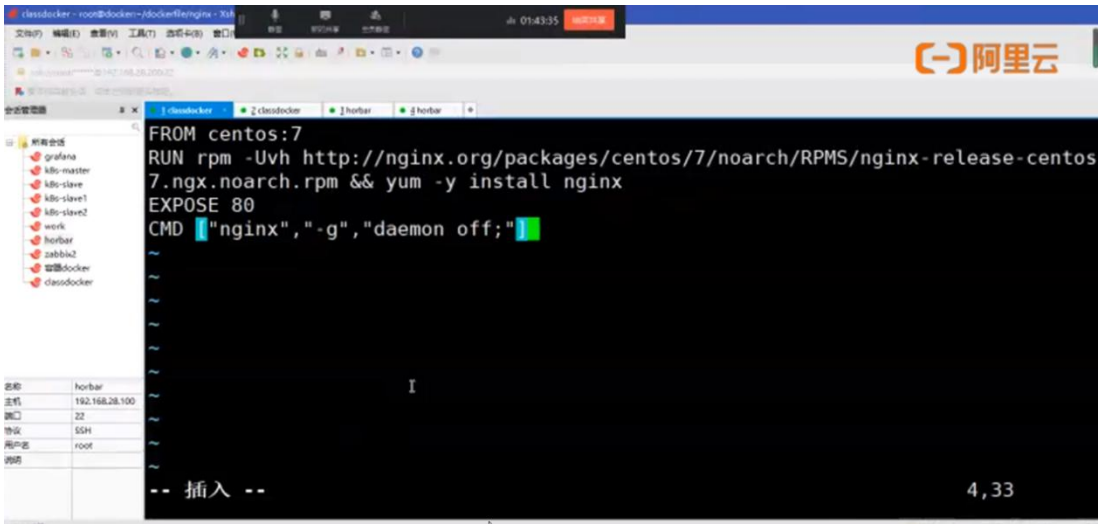
http://nginx.org/packages/centos/7/noarch/RPMS/

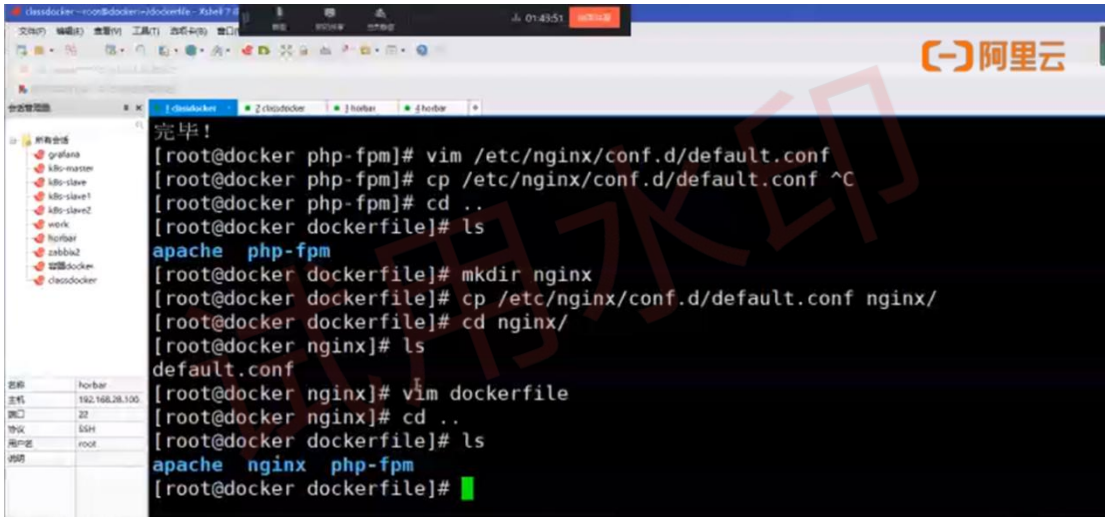
nginx-release-centos-7-0.el7.ngx.noarch.rpm

RUN yum -y install nginx

EXPOSE 80

CMD ["nginx","-g","daemon off;"]





阿里云 开发者训练营

构建和启动

构建镜像:

```
cd /root/nginx
docker build -t nginx:v1 .

cd /root/php-fpm
docker build -t php-fpm:v1 .
```

启动容器:

```
docker run -itd -p 8800:80 \
-v /root/nginxweb/:/usr/share/nginx/html/ \
--name nginx  nginx:v1

docker run -itd --network=container:nginx \
 -v /root/nginxweb/:/usr/share/nginx/html/ \
php-fpm:v1
```

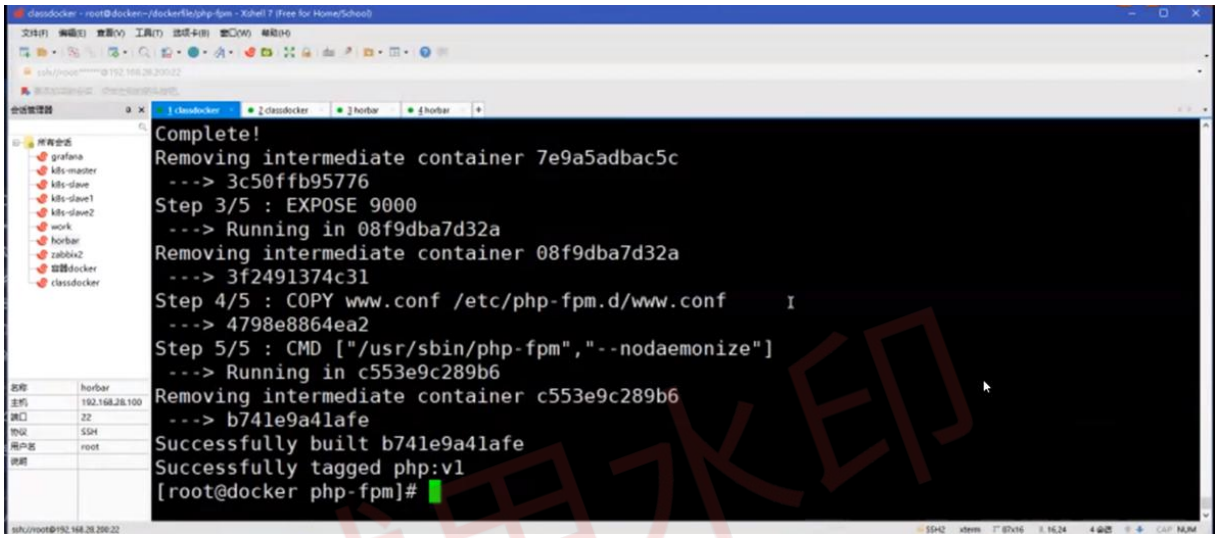
5.构建镜像

cd /root/nginx

docker build -t nginx:v1 .

cd /root/php-fpm

docker build -t php-fpm:v1 .



6.启动容器

docker run -itd -p 8800:80 \

-v /root/nginxweb/:/usr/share/nginx/html/ \

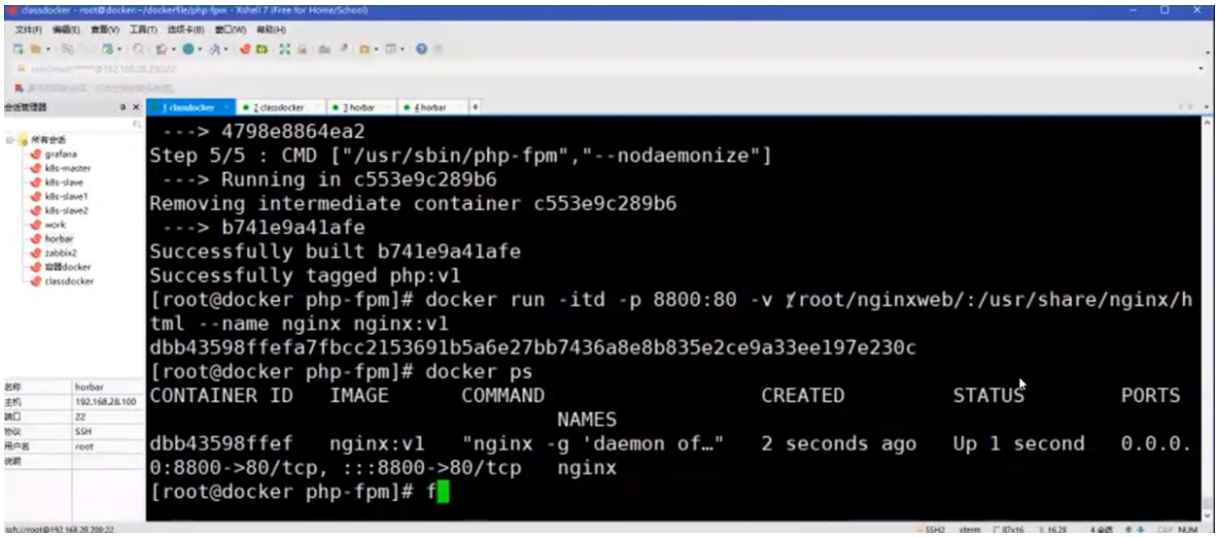
--name nginx nginx:v1

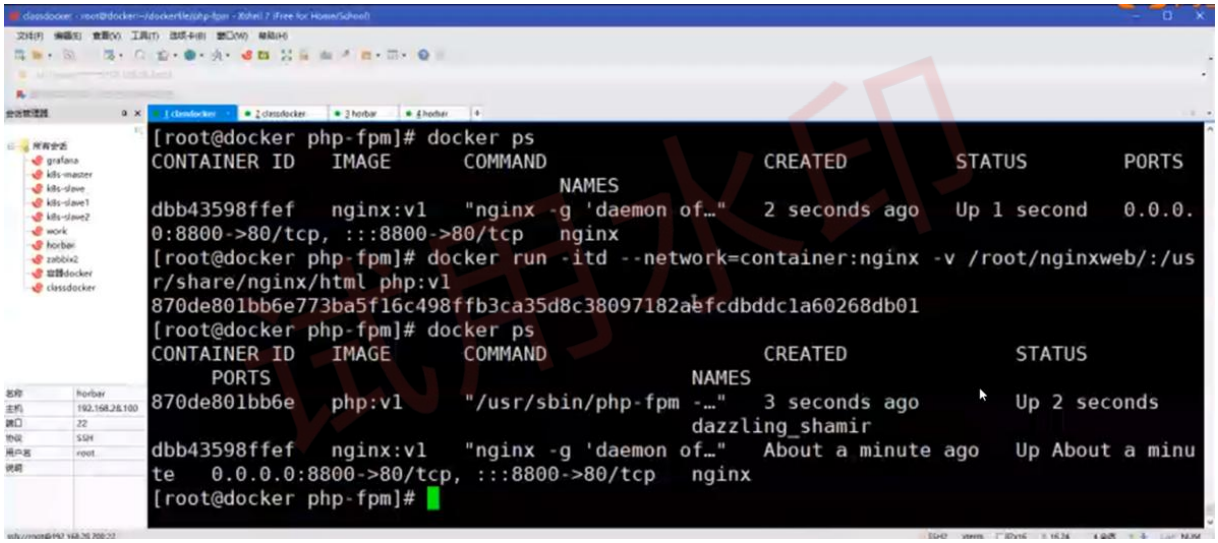
docker run -itd --network=container:nginx \

-v /root/nginxweb/:/usr/share/nginx/html/ \

php-fpm:v1

试用水印





7.验证

直接网页打开即可。